



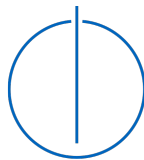
DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

View Matching on Query Graphs

Philipp Klocke





DEPARTMENT OF INFORMATICS

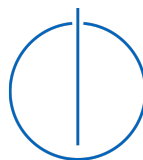
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

View Matching on Query Graphs

View Matching auf Anfragegraphen

Author:	Philipp Klocke
Supervisor:	Prof. Dr. Thomas Neumann
Advisor:	Michael Freitag, M.Sc.
Submission Date:	15.3.2022



I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 15.3.2022

Philipp Klocke

Acknowledgments

I want to thank my Michael for the continuous support and contributions as well as countless constructive discussions during my Masters Thesis. Thanks also to the rest of the Database chair, especially Prof. Neumann who was very approachable for valuable input and Per Fuchs, who picked up a lot of nitpicks from my intermediate presentation and provided helpful feedback.

Also, a big thank you to all my friends who supported me with food and memes, the modern “panem et circenses”, during the stressful last weeks.

Abstract

Materialized views can significantly improve query execution times. While this is especially true in the presence of large aggregations, also avoiding computation of smaller expensive predicates is favorable. However, using materialized views does not come for free and there are several issues that must be addressed. Views have to be created, for example by a view selection algorithm, maintained to store the most recent version of the data and lastly applied to incoming queries.

In this work, we focus on the applicability of views on incoming queries, the so called view matching problem. Given a query and all materialized views in a database system, the optimizer should find the optimal execution plan, utilizing views if it is beneficial. Therefore, the database system needs to decide which views match, and how to insert them into the query. Most existing algorithms restrict the problem to find a single matching view that can answer a query. In this work, we present an algorithm that is tailored to partial view matching, that is replacing parts of a query with a matching view. Our approach reduces view matching to a graph overlay problem between two extended query graphs. Thus, we can re-use the query graphs from join ordering and show how a scalable solution can be implemented in an actual database management system. Furthermore, we demonstrate how to extend the query graph, to match views with outer-joins and arbitrary operators. We prove that the algorithm scales in linear time with respect to query and view size, and show experimental results for our claims.

Finding an optimal execution plan with views requires an integration of the view matching into the optimizer. Thus, we also present how to apply our view matching algorithm in a DP-based plan generator.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
2. Background	3
2.1. Relational Algebra	3
2.1.1. Normal Form for SPJ Queries	4
2.1.2. Outer Joins	5
2.1.3. Join-Disjunctive normal form for SPOJ Queries	5
2.2. Query Trees	7
2.3. Join Ordering Problem	7
2.4. Query Graphs	9
2.4.1. Rebuilding a Join Tree	9
2.4.2. Outer Joins in Query Graphs	10
2.5. Materialized Views	11
2.5.1. View Selection	12
2.5.2. View Maintenance	12
2.5.3. View Matching	12
3. Design	16
3.1. View Matching on Query Trees	16
3.2. View Matching on Query Graphs for SPJ Queries	20
3.2.1. Extending the Query Graph	20
3.2.2. Matching the View	21
3.2.3. View Insertion	24
3.3. Refining the Matching Criteria	25
3.3.1. Definition of Matching Criteria	26
3.3.2. Cardinality Criterion	27
3.3.3. Tuple Inclusion Criterion	27
3.3.4. Attribute Inclusion Criteria	29
3.3.5. Finding the Residual Predicate	30

3.3.6. Full Algorithm	30
3.4. Outer Joins (SPOJ)	33
3.4.1. Nullification	33
3.4.2. Join degradation	38
3.5. Non-SPOJ Operators	38
3.5.1. Connecting Operators and SPOJ-Components	40
3.5.2. View Matching with Connecting Operators	41
3.6. Choice of matching view	42
3.6.1. Cost Function for Views	42
3.6.2. Integrating View Matching into the Optimizer	43
4. Implementation	44
4.1. Data Structures	44
4.1.1. Unique Identifiers	44
4.1.2. Set Inclusion and Equality	45
4.1.3. Connecting Operators	46
4.2. Predicate Matching	46
4.3. View Insertion	46
4.4. Shortcomings	47
5. Evaluation	49
5.1. Large Views	49
5.2. Multiple partially-matching Views	50
5.3. Non-matching Views	53
6. Future Work	55
7. Related Work	57
8. Conclusion	59
A. Proofs	60
List of Figures	64
Bibliography	65

1. Introduction

Recent years have brought rapid growth to data warehousing systems and other database management systems that handle tremendous amounts of data. Not only the amounts of data held and queried is increasing, but also the size of each individual query. Business intelligence tools can generate queries that reference up to about 4,000 relations [1], so scalability for large queries is becoming more and more important.

A tool that can help with both small and large queries are materialized views. Materialized views can be used to store any query result in an automatically maintained state that can be queried without computational overhead. Ideally, large parts of a query can be answered by a materialized view to reduce required computations, but even a materialized view that holds one or two expensive joins or other operations can drastically reduce query computation time. It is thus important to check whether a view can be applied to a query. With respect to query compilation time, this check should be fast and also scale up to thousands of relations, joins and other operations.

However, speed is always in conflict with completeness and there is a fine line between a fast and simple view matching approach with a high rejection rate and a NP-complete [2] algorithm that will itself take more time than query compilation and execution but never rejects a matching view. Solutions for view matching exist, but they are either only considering queries that consist of Selection, Projection and Join (SPJ) operators, or overly complex to implement. Even with an algorithm that decides whether a view matches, the optimizer and plan generator should be aware of view matching and consider plans where parts of a query are substituted by a view.

Our contributions to this problem are as follows. We design a view matching algorithm that can match views and queries with SPJ operators based on a query graph representation. By reducing the problem of view matching to a graph overlay problem, the algorithm scales in near linear time with respect to query size, defined by the amount of input relations and joins. We sketch an extension of this algorithm to outer-join operators and non-SPOJ operators. Our approach is extensible for any operator that can be expressed in a query graph, and can be fine-tuned between precision and complexity. Furthermore, we discuss how to integrate a view matching algorithm into a dynamic-programming based plan generator to generate view-aware optimal evaluations plans. We will discuss how our algorithm can be implemented efficiently and then show that the implementation indeed scales in close-to linear fashion with respect to query size.

The rest of this document is structured as follows: We start by introducing relevant theoretical background in Chapter 2. Then, we construct up our view matching algorithm step by step in Chapter 3. In Chapter 4 we discuss implementation details that are used to retain the low overhead of our algorithm. We show the time complexity in Chapter 5 and discuss further enhancements in Chapter 6. Lastly, we discuss how our work relates to other current work in Chapter 7.

2. Background

We first need to define some terms regarding view matching in database systems before we present our work. We start by giving standard definitions for relational algebra, including outer joins and discuss normal forms of expressions in relational algebra. Next, we discuss the join-ordering problem and introduce query graphs as a practical tool for solving this problem. Lastly, we explain materialized views, focusing on the view matching problem and presenting existing solutions and their drawbacks.

2.1. Relational Algebra

We will be using relational algebra throughout the thesis, so we will give a quick overview over the notation here. We first introduce the standard relational algebra and extend it to bag relational algebra which represents the evaluation model of our database system. For SPJ queries, that is queries that only contain selections, projections and joins, we provide a normal form and afterwards, discuss, how relational algebra with outer joins (SPOJ queries) can be normalized.

Relational algebra works on tuples and relations. Tuples are $t = (t_a, t_b, \dots)$, where each element has a name $a, b, \dots$, so the position of elements in a tuple is irrelevant. The element names are called attributes, and the attributes of a tuple are its schema $\mathcal{S}(t) = \{a, b, \dots\}$. We define $r \cup s = (r_a, \dots, r_n, s_{a'}, \dots, s_{m'})$ as the extension of r by s , requiring that their schemas are disjunct. The resulting schema $\mathcal{S}(r \cup s)$ will be $\mathcal{S}(r) \dot{\cup} \mathcal{S}(s) = \{a, \dots, n, a', \dots, m'\}$.

A set of tuples that share the same attributes is called relation, and we assume that attributes of different relations are always disjunct. Operators take on one or multiple relations as input and output an new relation. The operators that will be most important for our work are projection Π , selection σ , (inner-) join $\bowtie$ and outer joins ($\Join$, $\ltimes$, $\Join$ which we will introduce later).

Their formal definitions are:

$$\Pi_\beta(R) = \{t_\beta \mid t \in R\} \quad (2.1)$$

$$\sigma_p(R) = \{t \mid t \in R \wedge p(t)\} \quad (2.2)$$

$$R \times S = \{r \cup s \mid r \in R \wedge s \in S\} \quad (2.3)$$

$$R \bowtie_p S = \sigma_p(R \times S) \quad (2.4)$$

Up to this point, all operators except for projection work the same if we use bag semantics, that is defining relations as multisets of tuples. Selection works on each individual tuple, so we can keep it as is. The cross product will have to respect cardinalities of each tuple in each relation. The (inner) join is defined via selection and cross product so it does not have to be changed. The projection operator however will have to be redefined to not remove duplicates such that $\Pi_{\{a\}} \{ |(1_a, 1_b), (1_a, 2_b)(2_a, 1_b)| \} = \{ |(1_a), (1_a), (2_a)| \}$.

2.1.1. Normal Form for SPJ Queries

A normal form for SPJ queries looks like $\Pi_\beta \sigma_p (R \times S \times \dots)$. To prove this, it suffices to show that joins and cross products are associative and commutative, projections and selections can be moved in front of cross products and projections can be moved in front of selections. Also, multiple projections and multiple selections have to be unified respectively.

Since the order inside a tuple does not matter, cross product and join are commutative and associative:

$$R \times S = S \times R \quad (2.5)$$

$$R \bowtie_p S = S \bowtie_p R \quad (2.6)$$

$$R \times (S \times T) = (S \times R) \times T \quad (2.7)$$

$$R \bowtie_p (S \bowtie_q T) = (R \bowtie_p S) \bowtie_q T \quad (2.8)$$

Projections and selections can be unified respectively:

$$\Pi_\beta \Pi_\gamma R = \Pi_{\beta \cap \gamma} R \quad (2.9)$$

$$\sigma_p \sigma_q R = \sigma_{p \wedge q} R \quad (2.10)$$

Projections and selections can be moved out of cross products and joins:

$$(\Pi_\beta R) \times (\sigma_q S) = \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q (R \times S) \quad (2.11)$$

$$(\Pi_\beta R) \bowtie_p (\sigma_q S) = \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q (R \bowtie_p S) \quad (2.12)$$

Projections can be moved in front of selections:

$$\sigma_p (\Pi_\beta R) = \Pi_\beta (\sigma_p R) \quad (2.13)$$

Proofs for these claims can be found in Appendix A.

To build the normal form we first replace every join $R \bowtie_p S$ with its definition $\sigma_p (R \times S)$ according to Equation (2.4) and then repeatedly pull up all projections and selections by using Equation (2.11). We will end up with something like $\Pi_\beta \sigma_p \sigma_q \Pi_\gamma \dots (R \times S \times \dots)$. Now re-order the projections and selections to $\Pi_\beta \Pi_\gamma \sigma_p \sigma_q \dots (R \times S \times \dots)$ and lastly, apply Equation (2.9) and Equation (2.10) to obtain $\Pi_{\beta \cap \gamma \cap \dots} \sigma_{p \wedge q \wedge \dots} (R \times S \times \dots)$. The first step (replacing joins with cross products) can be omitted to obtain $\Pi_{\beta \cap \gamma \cap \dots} \sigma_{p' \wedge q' \wedge \dots} (R \bowtie_{p'} S \bowtie_{q'} \dots)$ if the original expression does not contain cross products.

2.1.2. Outer Joins

There is one group of operators missing which are the outer joins. For simplicity we will only introduce the left outer join $R \bowtie_p S$. It takes all tuples from R and extends the ones that find a matching partner in S like the inner join. Tuples from R that don't find a match will be null-extended to $\mathcal{S}(R) \cup \mathcal{S}(S)$. Let NULL be the tuple $(null, \dots)$ with schema $\mathcal{S}(S)$, then we can define left outer join in set-semantics:

$$R \bowtie_p S = R \bowtie_p S \cup \{r \cup \text{NULL} \mid r \in (R \setminus \Pi_{\mathcal{S}(R)} (R \bowtie_p S))\} \quad (2.14)$$

By using the appropriate set-minus operation for multisets, the definition can be extended to multisets as well, but this is out of scope.

The outer join is not associative. Let $R = \{|(1_r)|\}$, $S = \{|(2_s)|\}$, $T = \{|(1_t)|\}$, then:

$$(R \bowtie_{r=s} S) \bowtie_{s=t} T = \{|(1_r, null_s)|\} \bowtie_{s=t} T = \emptyset$$

Changing the order of operations changes the result:

$$R \bowtie_{r=s} (S \bowtie_{s=t} T) = R \bowtie_{r=s} \emptyset = \{|(1_r, null_s, null_t)|\}$$

Thus, we can not build a normal form as easily as for SPJ queries.

2.1.3. Join-Disjunctive normal form for SPOJ Queries

Galindo-Legaria showed that a normal form for SPOJ queries exists, which is based on minimum unions.

Let R_1 and R_2 be relations with schemas $\mathcal{S}(R_1) = \mathcal{S}_1$ and $\mathcal{S}(R_2) = \mathcal{S}_2$. We define the following operations:

2. Background

- $R_1 \uplus R_2$ the so called outer union first null-extends the tuples of each relation to $\mathcal{S}_1 \cup \mathcal{S}_2$ and then takes the union of the results (without duplicate deletion).
- $R \downarrow$ removes all subsumed tuples in R . A tuple t is subsumed iff. there exists another tuple in R that matches the non-null values of t but has fewer null entries.
- $R_1 \oplus R_2 = (R_1 \uplus R_2) \downarrow$ is the minimum union. It is commutative and associative, and has lower precedence than joins.

Then, a normal form can be built from the following equivalences [3, 4]:

If $R_1 = R_1 \downarrow$ and $R_2 = R_2 \downarrow$

$$R_1 \bowtie_p R_2 = R_1 \bowtie_p R_2 \oplus R_1 \quad (2.15)$$

$$R_1 \bowtie R_2 = R_1 \bowtie_p R_2 \oplus R_1 \oplus R_2 \quad (2.16)$$

If $R_3 = R_3 \downarrow$:

$$(R_1 \oplus R_2) \bowtie_p R_3 = R_1 \bowtie_p R_3 \oplus R_2 \bowtie_p R_3 \quad (2.17)$$

If p' references only R_1 :

$$\sigma_{p'}(R_1 \bowtie_p R_2 \oplus R_2) = (\sigma_{p'} R_1) \bowtie_p R_2 \oplus (\sigma_{p'} R_1) \quad (2.18)$$

If p' is strong (that is it evaluates to false for any null-value) and references only R_1 :

$$\sigma_{p'}(R_1 \bowtie_p R_2 \oplus R_2) = (\sigma_{p'} R_1) \bowtie_p R_2 \quad (2.19)$$

By repeatedly applying these rules, an arbitrary query can be transformed into join disjunctive normal form, which is a repeated minimum union over multiple terms that only include inner joins and selections. For example:

$$\begin{aligned} R_1 \bowtie_{p_{12}} R_2 \bowtie_{p_{23} \sigma_{p_3}} R_3 &= (R_1 \bowtie_{p_{12}} R_2 \oplus R_2) \bowtie_{p_{23} \sigma_{p_3}} R_3 \\ &= (R_1 \bowtie_{p_{12}} R_2 \oplus R_2) \bowtie_{p_{23} \sigma_{p_3}} R_3 \\ &= (R_1 \bowtie_{p_{12}} R_2 \bowtie_{p_{23}} R_3) \oplus R_2 \bowtie_{p_{23} \sigma_{p_3}} R_3 \\ &= (R_1 \bowtie_{p_{12}} R_2 \bowtie_{p_{23}} R_3) \oplus \sigma_{p_3} R_2 \bowtie_{p_{23}} R_3 \end{aligned}$$

Each of the terms in the minimum-union-disjunct in itself can be transformed into SPJ normal form, as presented in Section 2.1.1.

Some more useful associative identities for outer joins follow from the join-disjunctive normal form. We only show some of them, for the full set refer to [5]:

$$(R_1 \bowtie_{p_{12}} R_2) \bowtie_{p_{23}} = R_1 \bowtie_{p_{12}} (R_2 \bowtie_{p_{23}} R_3) \quad (2.20)$$

$$(R_1 \bowtie_{p_{12}} R_2) \bowtie_{p_{23}} = R_1 \bowtie_{p_{12}} (R_2 \bowtie_{p_{23}}) \quad (2.21)$$

If p_{23} rejects nulls on $\mathcal{S}(R_2)$:

$$(R_1 \bowtie_{p_{12}} R_2) \bowtie_{p_{23}} R_3 = R_1 \bowtie_{p_{12}} (R_2 \bowtie_{p_{23}} R_3) \quad (2.22)$$

If p_{12} and p_{23} reject nulls on $\mathcal{S}(R_2)$:

$$(R_1 \bowtie_{p_{12}} R_2) \bowtie_{p_{23}} R_3 = R_1 \bowtie_{p_{12}} (R_2 \bowtie_{p_{23}} R_3) \quad (2.23)$$

These equivalences will be helpful later, when we talk about the join ordering problem.

2.2. Query Trees

Query trees (also called operator trees) are a representation for expressions in relational algebra. Most database systems use them as primary representation for queries and perform optimizations on them, so they are worth to introduce and explain. Furthermore, we describe how a query tree is evaluated and how a query is translated into a query tree.

In a query tree, every operator and relation form a node. The inputs of an operator are its children, so $R \bowtie S$ is represented as a tree with a join at the root. The join has two children, R and S which are leaf nodes. Base relations are always leaves in the query tree, and the output of the tree is the output of the topmost operator.

A query tree can be evaluated recursively by evaluating each individual operator with respect to its inputs. The leaf nodes aka base relations can simply be queried and loaded from storage.

The process of obtaining a query tree is depicted in Figure 2.1. The root node is formed by the **SELECT** statement, which is translated to projection. Next, all selection predicates stemming from the **WHERE** clause are applied. Last, all inputs from the **FROM** clause form a tree of cross products.

Evaluating cross products and filtering out tuples with selections later on is very inefficient. Thus, cross products with corresponding predicates (that filter the result by referencing both of the inputs to the cross product) are transformed into joins. Other predicates are pushed down as far as possible, to further reduce intermediate results.

2.3. Join Ordering Problem

A motivation to the join ordering problem is as follows: Assume that we have a query of $R_1 \bowtie_{p_{12}} (R_2 \bowtie_{p_{23}} R_3)$, where R_1 contains 1 tuple and R_2 and R_3 contain a million tuples each. Keeping the precedence as indicated by the parentheses would mean potentially computing an intermediate result of $R_2 \bowtie_{p_{23}} R_3$ with over a million tuples, just to reduce it to 1 tuple later on, when joining with T_1 . So, the preferred order of execution would be $(R_1 \bowtie_{p_{12}} R_2) \bowtie_{p_{23}} R_3$

2. Background

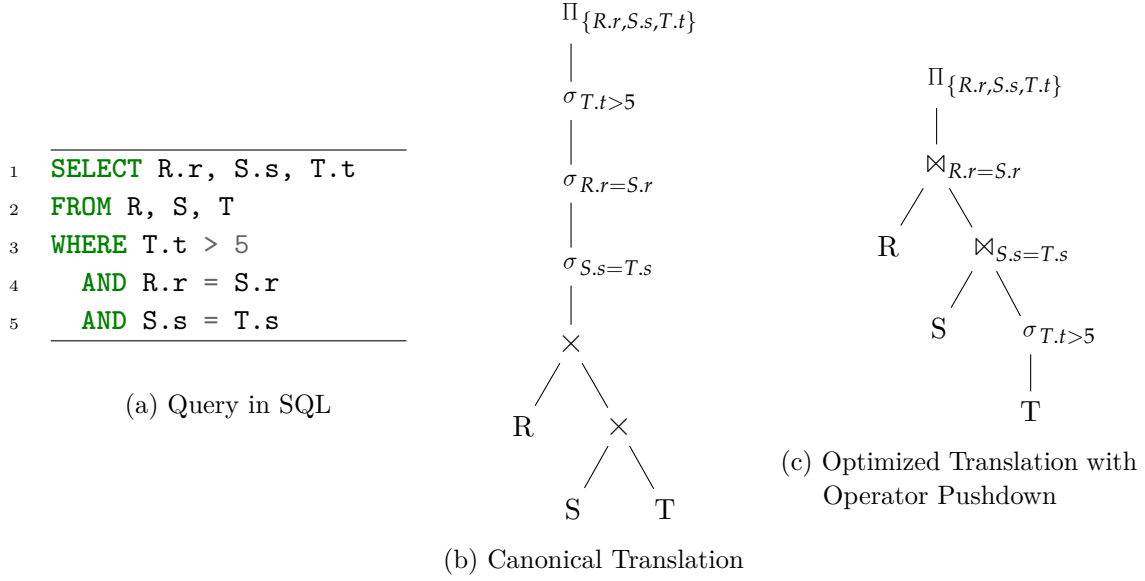


Figure 2.1.: Translating a Query to a Query Tree

We explained in Section 2.1, that some operators, especially inner joins, are associative, so there must be multiple join trees that express the same query but compute it differently. The join ordering problem is to find the best of them wrt. to a cost function.

In the following, consider query trees that only consist of joins and base relations, so called join trees. To derive a join tree from a query tree, all selections that are not transformed into joins can be pushed down into base relations (we will still pretend that they are base relations afterwards) or applied on top of the join tree. Selections also can be applied on top of the join tree. Further, we assert that there are no non-inner joins in the query tree.

A query tree is evaluated bottom-up, since the base relations can be queried and the operators on top of them have to be computed. Thus, all tuples from the base relations will be shoved upwards through the join tree. Every operator will access and operate on all tuples coming from its children and any operation takes time and potentially space, so it is beneficial to keep the amount of tuples as small as possible at any point. We define the amount of tuples in an evaluated tree T as $|T|_e$ and then use a simple cost function as follows [6, p. 34ff.].

$$\mathcal{C}(T) = \begin{cases} 0 & \text{if } T \text{ is a base relation} \\ |T|_e + \mathcal{C}(T_1) + \mathcal{C}(T_2) & \text{if } T = T_1 \bowtie T_2 \end{cases} \quad (2.24)$$

The cardinality of T can be estimated recursively, by where $f_{ij} \leq 1$ is the selectivity of

the join predicate between R_i and R_j .

$$f_{ij} = \frac{|T_i \bowtie_{p_{ij}} T_j|_e}{|T_i \times T_j|_e} \quad (2.25)$$

$$|T|_e = \begin{cases} |R|_e & \text{if } T \text{ is base relation } R \\ \prod_{R_i \in T_1, R_j \in T_2} f_{ij} \cdot |T_1|_e \cdot |T_2|_e & \text{if } T = T_1 \bowtie T_2 \end{cases} \quad (2.26)$$

2.4. Query Graphs

Most approaches to solve the join ordering problem rely on query graphs. They transform a join tree into a query graph, search this graph for an optimal ordering and then rebuild the optimal join tree from the graph. In the following, we present the standard definition of a query graph, how it can be constructed. Afterwards, we show how to rebuild a query tree from a query graph and discuss how it can be extended to outer joins.

A query graph consists of a set of vertices and edges $G = (V, E)$. Commonly, every vertex stands for a base relation and every edge is undirected and connects two relations, so it can be represented by an unordered pair. An edge between A and B is inserted when there exists an inner join with a join predicate p_{AB} that references A and B . The edge is then labeled with the corresponding predicate [7]. We define $p(e)$ to return the predicate for an edge.

2.4.1. Rebuilding a Join Tree

To give an example how to rebuild a join tree from a query graph, we sketch an approach based on union-find. The approach starts by selecting any edge and the vertices that it connects. The choice of edge is arbitrary, since inner joins are commutative as discussed in Section 2.1. Let that edge be (i, j) , labelled with predicate p_{ij} , then term it represents is $i \bowtie_{p_{ij}} j$. We create the join tree that holds this term, and store it in a new vertex x . The old vertices can subsequently be removed and any edge referencing them will be rerouted to x . If there was a vertex k with edges (k, i) and (k, j) , these edges will have to be merged into a new edge (k, x) with associated predicate $p((k, x)) = p((k, i)) \wedge p((k, j))$. In the example in Figure 2.2, this happens in the second step, when the edge (U, V) is compressed. The edges (U, R) and (V, S) have to be merged together. This procedure is repeated until only one node is left, which then holds the rebuilt join tree. By choosing the order of picked edges carefully, an algorithm can minimize the costs for evaluating the join tree as defined in Section 2.3.

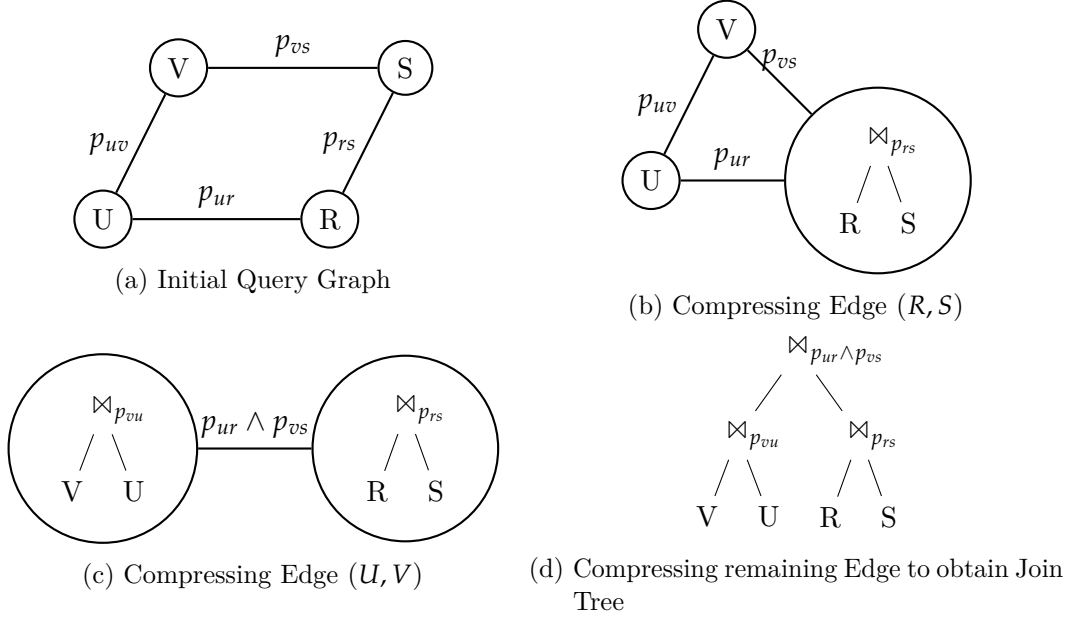


Figure 2.2.: Converting a Query Graph into a Join Tree

2.4.2. Outer Joins in Query Graphs

The most natural extension of query graphs to outer joins would be to introduce new directed edges that represent outer joins. The arrow tip indicates the side where nulls might be introduced. We present the query graph for $R \bowtie_{p_{rs}} (S \bowtie_{p_{st}} T)$ in Figure 2.3. This query graph is ambiguous, since we can construct the join tree for $(R \bowtie_{p_{rs}} S) \bowtie_{p_{st}} T$ from the query graph by applying the previously presented algorithm [3]. As presented in Section 2.1.2, this reordering is not allowed and changes the output of the query. This means, that the naturally extended query graph does no longer capture the semantics of the query.

The solution is to restrict that an edge can only be picked when its required inputs are present [8, 9]. This property can be encoded in a hypergraph, where an edge is a tuple of sets of vertices. Tuples can still be left unordered, but it has to be remembered whether an edge was a left-, right- or full outer join. We indicate that by giving the edge a direction, but still talk about unordered tuples.

The choice of edge in above procedure is now restricted to edges whose sides are all merged together already. In the case of $R \bowtie_{p_{rs}} (S \bowtie_{p_{st}} T)$, the inner join represented by (S, T) has to be picked before the outer join. Figure 2.4 shows the hyperedge (R, S, T) that enforces this behavior on the left side. Special care has to be taken when to insert a hyperedge, to not overly restrict the query graph. The right hand side of the figure shows

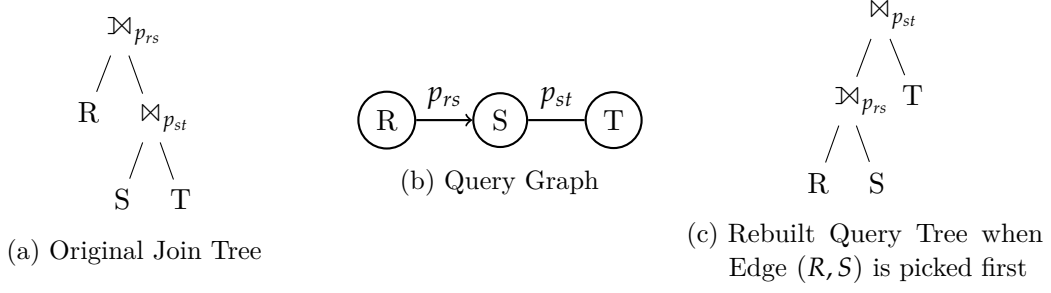


Figure 2.3.: Ambiguity of Natural Extension of a Query Graph with Outer Joins

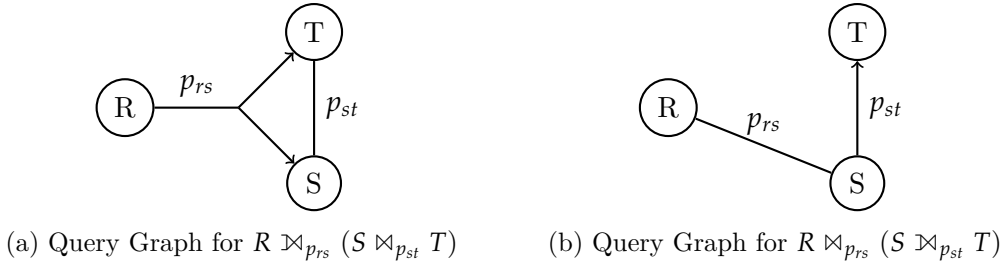


Figure 2.4.: Extension of a Query Graph by Outerjoin re-ordering Constraints as Hyperedges

the query graph for $R \bowtie_{p_{rs}} (S \bowtie_{p_{st}} T)$, which is indeed re-orderable to $(R \bowtie_{p_{rs}} S) \bowtie_{p_{st}} T$, as presented in Section 2.1.3. Nearly all outer join re-ordering constraints and equivalences can be expressed in terms of hypergraphs [8].

2.5. Materialized Views

Let us now introduce the idea of a view. A view in general is just a stored query, that can be used in other queries as if it was a base table or a common table expression (**with** statement). While some database systems restrict views to a limited subset of SQL [10], we consider views to be an arbitrarily complex **SELECT** query.

Views themselves are a good tool for managing query complexity and access rights, or handling the requirements of multiple database users [11]. However, they can also be materialized, such that the result of the view query is stored physically. In this case, they significantly speed up query execution times, because the respective view query does no longer need to be evaluated and can instead be read from the materialized table. From now on whenever we write “view” we refer to a materialized view. Unfortunately, the speedup does not come for free since materialized views introduce several problems. They have to be created, maintained and applied. We highlight all of these problems

briefly and focus on the application of views, the so called view matching problem that this work is about.

2.5.1. View Selection

While views of course can be created manually, some specific views could highly speed up a given query workload. View selection tries to find and create these views automatically [10], with the goal to improve query performance by lowering processing time. However, each materialized view will have to be maintained so the maintenance costs should be kept low and the views must not exceed the available storage space. So, the problem is to choose views that have a desirable balance between processing cost, maintenance cost and storage space. It is known to be NP-complete [12], thus different view selection methods have been designed for approximation [13].

2.5.2. View Maintenance

View Maintenance is the problem of maintaining an up-to-date state of a materialized view, while the base relations change. Maintenance can happen immediately after a base relation is changed, or be deferred to a later stage. In most cases it is cheaper to incrementally update a view than to recompute it entirely and further optimizations exist regarding on the complexity of a view statement [14, 15].

2.5.3. View Matching

After view selection algorithms (or database system users) created views and view maintenance held them up to date, the views can be put to use. View matching, sometimes called “view exploitation” [16], takes a query and tries to find the best execution plan utilizing the materialized views in a database system. Different dimensions of this problem exist, for example the resulting plan could be restricted to only a single view or a combination of views but without base tables [17]. Another definition is that materialized views only replace parts of the query. We call this partial view matching and views are used to replace potentially costly computations in the query. Therefore, two questions need to be answered. Firstly, it has to be determined which views match parts of the query and secondly the best possible plan using these views must be found.

View Matching by Goldstein and Larson

The former problem could be solved by syntactic matching, that is checking if the query string and view string are equal. This however, would reject views that contain more

tuples than the query, which could be filtered out later. So, Goldstein and Larson defined criteria that a view has to fulfill to be used in a query [16].

- All rows from the query are present in the view
- All attributes that the query requires for selections are computable from the views output
- All attributes that the query outputs are computable from the views output
- All rows in the view occur with the correct duplication factor

Based on these criteria, they designed a view matching algorithm for SPJ queries with a single aggregation (SPJG). While they did not design their algorithm for partial view matching and instead try to answer a query from a single view, their matching criteria are applicable to partial view matching as well. Larson and Zhou extended the algorithm from SPJG to SPOJG queries and views by using the join distinctive normal form. In the following we will quickly explain their approach and then highlight shortcomings that we try to overcome in this work.

View Matching by Larson and Zhou

As discussed, SPOJ queries can be represented in join disjunctive normal form, that is $\mathcal{T}_1 \oplus \mathcal{T}_2 \dots$, where $\mathcal{T}_i$ are SPJ queries in normal form. The algorithm first transforms both the query and view into join disjunctive normal form. Next it checks containment of the view in the query, by checking if for every term $\mathcal{Q}$ in the queries normal form, there exists a term $\mathcal{V}$ in the views normal form that contains or subsumes all tuples from $\mathcal{Q}$. They do not only check if every tuple from $\mathcal{Q}$ is also in $\mathcal{V}$, because a matching tuple with less null attributes could exist in $\mathcal{V}$, and these attributes could later be set to null. To check if each tuple exists or is subsumed, they check if the attributes in the view are less restrictive than in the query. That is implemented by predicate implementation. For every predicate p_v in $\mathcal{V}$ there has to be a predicate p_q in $\mathcal{Q}$, such that $p_q \Rightarrow p_v$. They also ensure the correct duplication factor of every tuple, by checking for joins that might change cardinalities.

Next, the algorithm checks whether all terms in $\mathcal{Q}$ can be recovered from $\mathcal{V}$ that is if there is a residual predicate p_r that can turn the output from $\mathcal{V}$ to the output of $\mathcal{Q}$. This is possible if there is a unique key in $\mathcal{V}$, because then every tuple is uniquely identifiable. However they also present other tricks to perform this, if no unique key is available.

After the residual predicate is built, the algorithm checks if all attributes that are either required to evaluate the residual predicate or to compute the query output are available. This check is performed with respect to equivalence classes. If for example an

attribute q of the query does not exist in the view, but instead the predicate $q = v$ and attribute v are present in the view, then v can be renamed to q .

Lastly, if all checks pass, the query is substituted by the view plus renames and the residual predicate.

Drawbacks of the Algorithm

While the solution by Larson and Zhou is very extensive, there are some shortcomings in their approach that we want to highlight in the following.

First of all, their algorithm is not designed for partial view matching. While they claim that it can be extended to partial matching, there are some important questions, that must be answered.

For example, it is unclear how they would determine which part of $\mathcal{Q}$ a view would replace. Checking all possible combinations of terms in $\mathcal{Q}$ would take $\mathcal{O}(2^{|\mathcal{Q}|})$ time. Searching for all terms of $\mathcal{V}$ in $\mathcal{Q}$ seems to fall back to syntactic matching, or requires some sophisticated checks.

If this is solved, and $\mathcal{V}$ matches some terms $\mathcal{T}_i$ of $\mathcal{Q}$, then these terms could be replaced by $\mathcal{V}$. However, the query afterwards looks like $\mathcal{V} \oplus \mathcal{T}_j \oplus \dots$ so some minimum unions will have to be computed. The minimum union was a union, that needs to check every tuple for subsumption. In SQL that might be expressed with a subquery. For example if tuples from the query might be subsumed by $\mathcal{V}$ on the attribute x , the representation in SQL could look like Listing 1. Of course the query itself could potentially contain even more minimum unions. And while algorithms for query unnesting exist [18], or minimum unions can sometimes be avoided altogether as shown by Larson and Zhou, subqueries introduce quadratic complexity and thus do not scale well.

Also, the algorithm can only handle **SUM** and **COUNT** expressions in aggregations, and only a single aggregation in the view. And lastly, even with the full extended algorithm, it remains to choose the best plan utilizing matching views.

```
1  --  $\mathcal{V}$ 
2  SELECT *
3  FROM View
4  WHERE [ResidualPredicate]
5  UNION ALL
6  --  $\mathcal{T}_j$ 
7  SELECT *
8  FROM [RestOfQuery] Q
9  WHERE [QueryPredicates]
10 -- Subsumption check
11 AND NOT EXISTS (
12     SELECT *
13     FROM View V
14     WHERE [ResidualPredicate]
15     AND V.x IS NOT NULL
16     AND Q.a = V.a AND Q.b = V.b AND ...
17 )
```

Listing 1: SQL Example for a Minimum Union between a partially matched View and the Rest of the Query

3. Design

Let us take a step back and start off with a very simple algorithm for partial view matching. We will iteratively show the shortcomings of the approach and how to overcome them, starting with an algorithm that matches views on query trees. After discussing that query trees are restricted by the join-ordering problem, we will go over to use query graphs instead. We show that they are useful data structures on which view matching can be performed efficiently, by first re-implementing the tree algorithm but this time oblivious of the join ordering. Next, we will show ways how to limit the false negatives, by discussing the criteria under which a view matches a query. Afterwards, we extend the algorithm to handle outer joins and non-SPOJ operators and lastly, we shortly discuss how to integrate view matching in a cost-based optimizer.

3.1. View Matching on Query Trees

A query tree is a representation of a query that can be evaluated bottom-up. It follows from the canonical translation of a query with operator pushdown and other optimizations such as translation of cross-products into joins. The advantage of using query trees is that they are very simple and nearly every database management system will use a tree representation at some point, but as we will show they are not entirely suitable for view matching and lead to a high rate of false rejections due to their fixed operator ordering.

To show how they could be used to perform view matching on query trees, we implement a simple algorithm that first searches for a subtree of the query tree that is equivalent to the view and then replaces it with the view. Also, we will give a brief intuition how we could extend our algorithm to deal with the join-ordering problem for inner joins, before we conclude that we should not pursue query tree based approaches further.

We first define a straightforward equality check for query trees by checking equality of two nodes and then recursing into their children. The algorithm in section 3.1 is stopping early, so for two query trees A and B the run time is $\mathcal{O}(\min\{|A|, |B|\})$ assuming checking expression equality is constant and $|A|$ is the amount of operators and inputs in A . It is important to note while every operator tree that is equal by this metric will produce the same output, there are potentially infinitely many other operator trees that are not equal by our metric but also produce the same output.

```

1 bool isEqualTree(OperatorNode a, OperatorNode b) {
2     // sanity checks
3     if a == null || b == null
4         || length(a.children) != length(b.children)
5     {
6         return false;
7     }
8
9     // actual comparison
10    if a.expression != b.expression { return false; }
11    for i in range(0, length(a.children)) {
12        if !isEqualTree(a.children[i], b.children[i]) {
13            return false;
14        }
15    }
16    return true;
17 }

```

Listing 2: Simple tree equality check

Figure 3.1 shows a query and a view that partially matches the query, as well as the resulting query with the inserted view. We can see that the view does not match at the query root, so our algorithm would have to recurse the query tree and insert the query at the point where it fits. Partial view matching became the problem of finding a subtrees equivalent to the query. This algorithm checks for equivalence at any possible point, so for query tree Q and view V the run time is $\mathcal{O}(|Q| \cdot \min\{|Q|, |V|\})$, where $|T|$ is the amount of nodes in a tree. Assuming $|V| \leq |Q|$ (otherwise the view could not even match), this reduces to $\mathcal{O}(|Q|^2)$. Through some engineering effort, we could simplify this to linear time $\mathcal{O}(|Q|)$.

A bigger challenge remains when looking at the example in Figure 3.2. Here, the view matching algorithm will fail, because it can not detect the equivalence $C \bowtie_{CD} D \equiv D \bowtie_{CD} C$, or the structural equivalence of the trees under join-reordering constraints. Reversing the join direction could be supported reasonably, but checking join-tree equivalence is difficult, especially when regarding also outer joins. In other words, the algorithm can deal neither with the associativity, nor commutativity of joins and instead depends on the fixed join ordering that the operator tree represents.

This leads to many false negatives, since there exist many difficult join orderings and corresponding query trees from which we only need to find a single one where the view

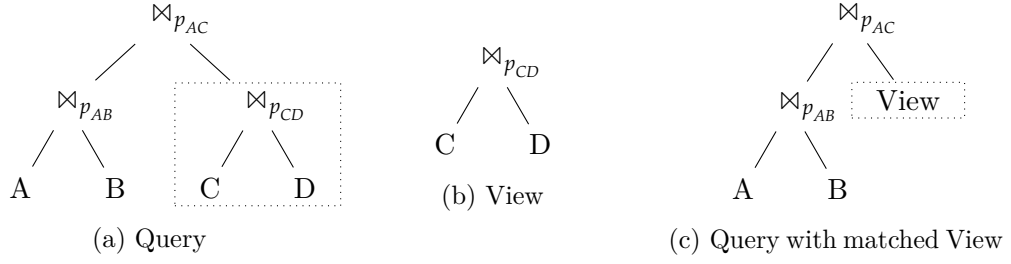


Figure 3.1.: Example of view matching on Query Trees

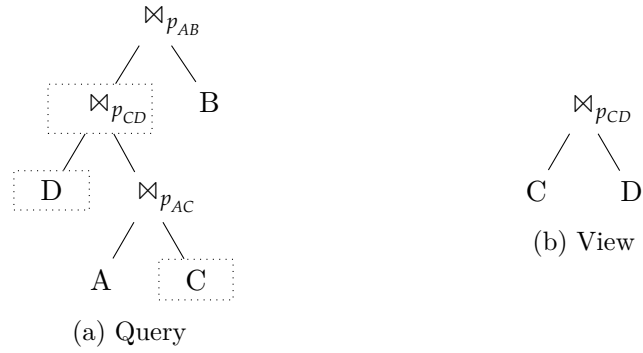


Figure 3.2.: Example of failing view matching on Query Trees

```

1  OperatorNode searchAndInsertView(
2      OperatorNode query_node,
3      OperatorNode view_root)
4  {
5      if isEquivalentTree(query_node, view_root) {
6          return view_root;
7      } else {
8          query_node.left = searchAndInsertView(query_node.left, view_root);
9          query_node.right = searchAndInsertView(query_node.right, view_root);
10         return query_node;
11     }
12 }

```

Listing 3: Inserting a view into a query tree

is a valid subtree. To overcome this limitation, we can abstract away from a fixed join ordering, instead finding a way how all possible join orderings can be checked.

To sketch a short idea how to achieve this in query trees, we give the following intuition. If we consider only inner joins, we could derive a normal-form of the query that looks like $\Pi\sigma(T_1 \bowtie_{p_1} T_2 \bowtie_{p_2} \dots \bowtie_{p_n} T_n)$ as shown in Section 2.1.1. Then, we would just have to check if all tables and joins from the view are included in the join-conjunct. If there are projections or selections in the view, we need to find their corresponding parts in the projection and selection operators of the normal-form. Transferring this idea back to the tree representation would mean to find all leaf nodes from the view tree in the query tree and then to traverse upwards to search all remaining operators from the view tree in the query tree. The intuition is that we would encounter all operators that operate on the inputs that the query shares with the view on the way up. In Figure 3.2 the inputs (C and D) and operators ($\bowtie_{p_{CD}}$) of the view are surrounded by dotted boxes in the query tree. We would need to take one step upwards from D and 2 steps upwards from C to find their corresponding join, thus seeing that all view inputs and operators are available, so the view would match in a different join order. After detecting that a view matches, we would still require a way to re-order the tree such that the query is insertable. While it is not obvious how to implement this in an efficient manner, especially wrt. to a cost-based optimizer (cf. Section 3.6), it would surely be possible to come up with a way for this.

However, the approach is overall bulky and only applicable to inner joins; other operators such as outer joins are not freely re-orderable and impose join re-ordering constraints on the query. To handle outer joins, we would need to extend the normal-form to the join disjunctive normal form of Galindo-Legaria and derive something similar the

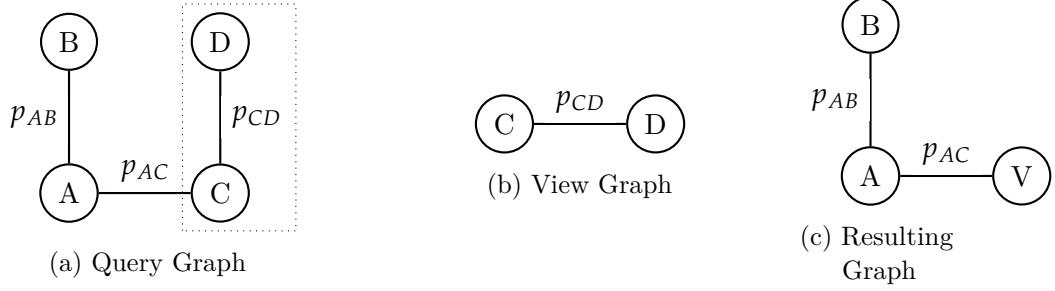


Figure 3.3.: A graph representation of the view matching that failed to match in the trivial query tree algorithm

approach of Larsson & Zhou that we presented in Section 2.5.3. Instead, we will go a different way and in the following present how we can check all possible join orderings by performing view matching on a query graph.

3.2. View Matching on Query Graphs for SPJ Queries

Query graphs are used to find an optimal join ordering, so they will be generated before join ordering and we do not have to consider additional overhead for computing the query graph. They encode all join re-ordering constraints, so they give a semantics which query trees are valid re-orderings but abstract away from a specific ordering.

A short motivation for using query graphs as input to view matching is given in Figure 3.3, where we show how the query and view from Figure 3.2 can be represented as query graphs. We created a vertex for every input to the query tree and for every join predicate p_{ij} referencing i and j , we inserted an edge between i and j . Highlighted by the dotted rectangle, there is a subgraph that is equal to the view graph, so we could replace it by the view.

In the following, we will formalize this and show how to use a query graph as input to the view matching problem. So far, query graphs do not handle selections or projections, so we will first extend them and then show how a simple algorithm like we used for the query trees can be implemented on query graphs.

3.2.1. Extending the Query Graph

In Section 2.4 we considered query graphs for join trees which only hold joins and base relations. However to match views, we also have to handle selections and projections. The latter can be stored outside of the query graph, since it only filters attributes from the output of a join tree that is built from the query graph.

To include arbitrary selections, we have to introduce additional edges. Previously we defined edges to be unordered tuples of two sets of vertices with associated predicate and join-type. An arbitrary selection predicate could access arbitrarily many relations, so we have to relax the definition of an edge. A selection $\sigma_p R$ on a base relation will form a self-loop ($\{R\}$) annotated with the predicate p . A (degenerate) selection that accesses multiple base relations $\sigma_{p_{rst}\dots} (R \circ S \circ T \circ \dots)$ where $\circ$ can be any join or cross product will translate to a hyperedge ($\{R\}, \{S\}, \{T\}$) with associated predicate $p_{rst}\dots$. Note that any selection on multiple inputs could also be a join: $\sigma_{a+b=c} (A \times B \times C)$ could also be expressed as a join with 3 inputs A , B and C and the join predicate $a + b = c$. So we now have three different types of edges:

- Regular edges are tuples with 2 elements each holding a single vertex, for example ($\{R\}, \{S\}$).
- Hyperedges that restrict the join ordering are tuples with 2 elements holding arbitrarily many vertices, for example ($\{R\}, \{S, T\}$).
- Edges for selections are tuples with arbitrarily many sets of single vertices and could be self loops, regular edges or hyperedges, for example ($\{R\}$), or ($\{R\}, \{S\}, \{T\}$). We will treat these exactly like regular edges and adapt the rebuild procedure accordingly to merge all of their adjacent vertices.

All of them are associated with a predicate $p(e)$ and have a type $\text{type}(e)$ which is either regular predicate for inner join or selection or (left-, right-, full-) outer join. To rebuild the query tree we can apply the algorithm from Section 2.4.1, with the restriction that hyperedges for outer joins or complex selections may only be picked when their inputs are available. At this point they are reduced to a regular edge which can be handled as discussed. Lastly, we apply all projections that were tracked separately on top of the resulting join tree.

The output of a query graph is defined as the output of the evaluation of a query tree that can be built from the query graph. Since all query trees that can be built from a query graph have the same output (by definition of the query graph), the output of a query graph is well defined.

3.2.2. Matching the View

With the newly defined query graph we can now implement an algorithm to identify a subgraph of the query graph that is equal to the view graph and replace it by the view graph just as we did for query trees and subtrees. We will at first restrict the query graph to SPJ expressions, handling selections and (inner) joins in the graph and

tracking projections separately. In the next sections, we will further extend the matching algorithm to handle outer joins and even arbitrary operators.

We will first show that finding an equal subgraph is comparable to finding an equal subtree (cf. Section 3.1), but independent of the join order. Afterwards, we will implement the actual algorithm.

By equality between two graphs, we mean that all nodes and edges of the graphs have to be equal, thus, the output of both graphs will be equal as well. Note that there might be infinitely many other graphs that give the same output, for example when we have two single vertex graphs, with vertices R and S respectively. If R and S hold the same data, then the output of both graphs will be the same as well. This is an inherent problem with any approach to view matching and we will not go into it.

In the following we will talk about the query graph of the view and the query graph of the query, which might be confusing, so we will abbreviate them as view graph $G_V = (V_V, E_V)$ and query graph $G_Q = (V_Q, E_Q)$.

Remember that we were interested in finding a subtree of a query tree that is equal to the view tree. A query tree is encoded by a graph, iff. that graph can be rebuilt into the query tree. Potentially, the query graph could be rebuilt into multiple query trees with different join order – this was its’ purpose after all. If we find a subgraph of the query graph that encodes the view tree, then we can first rebuild this subgraph by picking the corresponding edges and then rebuild the rest of the query graph. By definition of our rebuild algorithm (cf. Figure 2.2) this has to result in a query tree where the view tree forms a subtree, since operators are always added on top of the existing tree. So we proved the existence of a join ordering where the view tree is a subtree of the query tree. Thus, we can replace the subgraph with the view, as we did in the query tree based approach.

It remains to show how to find a subgraph of the query graph that encodes the view tree. We know that all query trees that consist of the same operators and inputs will be translated into the same query graph regardless of their join orderings. Thus, we can transform the view tree into the view graph and see whether it is a subgraph of the original graph. Then this subgraph of the original graph encodes the view tree. We conclude that finding a subgraph that is equal to the view graph is the equivalent of our algorithm defined on query trees.

From here, we can define an algorithm that checks whether an equivalent subgraph exists. We will first test if all vertices from the view graph exist in the query graph. Next we need to identify the corresponding subgraph and then check if all its edges are equal to the views edges wrt. to their type and associated predicate. The whole algorithm is depicted in Section 3.2.2.

For a start, testing whether all vertices from the view graph are present in the query graph is a set inclusion operation that tests whether $V_V \subseteq V_Q$. Semantically, this is

equivalent to finding all base relations of the view in the query graph.

Next we have to define what the subgraph that we want to match actually is. We use the definition of induced subgraphs. The induced subgraph $G|_{V'}$ of $G = (V, E)$ is (V', E') with $E' = \{(u, v) \mid (u, v) \in E, u \subseteq V', v \subseteq u\} \cup \{(v) \mid (v) \in E, v \subseteq V'\}$. This means that E' contains all (hyper-)edges that only connect to nodes in V' or in other words lie completely in the induced subgraph. We require the union operation, because we included selections that refer only a single set of relations in our definition of query graphs.

By this definition, the subgraph that could match the view graph is then $G_Q|_{V_V} = G_X = (V_X, E_X)$. The rest of the query graph is thus $G_R = (V_R, E_R) = (V_Q \setminus V_X, E_Q \setminus E_X)$. This rest graph is not a well defined graph, since it also contains edges that connect edges from itself to the matching subgraph, meaning that not every vertex referenced by an edge is in V_R .

Since V_V contains all inputs of the view, any edge in the view can only connect vertices from V_V , or in other words predicates in the view can only access inputs from the view. This means, that all edges from the view would have to be included in the induced subgraph, $E_V \subseteq E_X$. To test for full equality however, we also have to check if $E_X \subseteq E_V$. Also, we have to verify that for any matching pair of edges $e_v = e_x$, $e_v \in E_V$ and $e_x \in E_X$ their type and predicate matches $p(e_v) = p(e_x) \wedge \text{type}(e_v) = \text{type}(e_x)$. Semantically this is equal to finding all matching pairs of operators in the subgraph and view graph and checking whether they are equal.

There is one check missing which is not encoded in the graph, which was the check for projections. Since we are only regarding a subgraph of the query graph and we said that projections are applied on top of any rebuilt query tree, the rebuilt subtree will have no projections. Thus, we simply check that the view also contains no projection operators. If all the previous checks were successful, the view matches a subgraph of the query.

Let us quickly go over the run time complexities step-by-step. Checking if $V_V \subseteq V_G$ is computable in $\mathcal{O}(|V_V|)$, by uniquely labeling vertices and then iterating over vertices in the view and testing the existence of each in the query graph. To compute the induced subgraph, we can check for each edge whether all its vertices are included in the subgraphs vertices. As we said, checking whether a set of vertices is included in another set of vertices takes linear time, so testing vertex inclusion for all edges is $\mathcal{O}(|E_Q| \cdot |V_Q|)$ in the worst case. However, the set of vertices that an edge references is semantically equal to the amount of base relation a predicate accesses. In practice, this set is very limited in size, so the complexity tends to go towards $\mathcal{O}(|E_Q| \cdot c)$, where c is a constant factor that can be dropped from $\mathcal{O}$. Lastly, testing $E_X \subseteq E_V$ and $E_V \subseteq E_Q$ can again be performed in linear time by computing a signature for each edge. For any edge, its correspondent in the other edge set can be looked up in constant time and assuming that checking type and predicate equality takes constant time, the resulting complexity is $\mathcal{O}(|E_V| + |E_X|)$.

Since $E_X \subseteq E_Q$ and assuming that $|E_V| \leq |E_Q|$ and this reduces to $\mathcal{O}(|E_Q|)$.

In total, the complexity is thus $\mathcal{O}(|V_V| + |E_Q| \cdot c + |E_V| + |E_Q|) = \mathcal{O}(|V_V| + |E_Q|)$.

```

1  bool containsEquivalentSubgraph( $G_Q$ ,  $G_V$ ) {
2      for  $v_v$  in  $V_V$  {
3          if ! $V_Q$ .contains( $v_v$ ) { return false; }
4      }
5
6       $G_X := G_Q$ .get_induced_subgraph( $V_V$ );
7
8      for  $e_v$  in  $E_V$  {
9           $e_q := G_X$ .find( $e_v$ );
10         if  $e_q$ .expression !=  $e_v$ .expression { return false; }
11     }
12
13     for  $e_q$  in  $G_X$ .edges {
14          $e_v := G_V$ .find( $e_v$ )
15         if  $e_v$ .expression !=  $e_q$ .expression { return false; }
16     }
17
18     return true;
19 }
```

Listing 4: Basic Query Graph Inclusion Test with Equality

3.2.3. View Insertion

After an equivalent subgraph is identified, the subgraph can be replaced by the view. We include the view as a single new vertex in the query graph and remap all edges that connect to the induced subgraph to connect to the view vertex instead. Afterwards, the the whole induced subgraph is removed. This is pretty similar to the merge step of the rebuild algorithm from Section 2.4.1. Figure 3.4 illustrates this process. Everything that is modified or removed is highlighted by dotted lines, while insertions are drawn in blue.

The removal and remap procedure needs to access every vertex and edge in G_X , to remove it and check every edge in the rest graph G_R whether it needs to be remapped. Assuming that checking and remapping an edge takes constant time, again with the argument that edges are limited in size, the procedure requires $\mathcal{O}(|V_X| + |E_X| + |E_R|)$ time. We know that $V_X = V_V$ and $E_X \cup E_R = E_Q$, so the resulting time complexity is

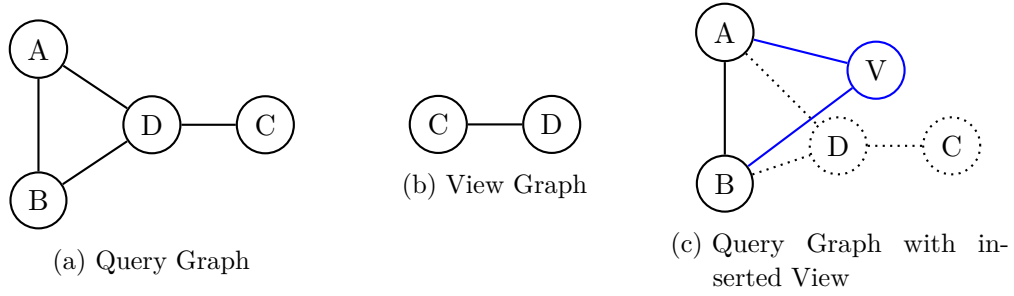


Figure 3.4.: Inserting a View Graph into a Query Graph

$$\mathcal{O}(|V_V| + |E_Q|).$$

3.3. Refining the Matching Criteria

So far, we have shown how to perform view matching independent of join ordering by using the query graph. However, we have only considered views that are exactly equal to the part of the query they replace, which highly restricts the applicability of views. Figure 3.5 shows a query graph that is not equal to the view graph. Even so, the structure roughly matches, and we could apply all edges and associated predicates that are not included in the view, later, on top of the view, as shown on the far right.

In the following, we will formalize a relaxation of the equivalence criterion, by formally defining when a view can be used to replace a part of the query. We then show how the new criteria can be checked by an algorithm and how a view that fulfills the criteria is inserted.

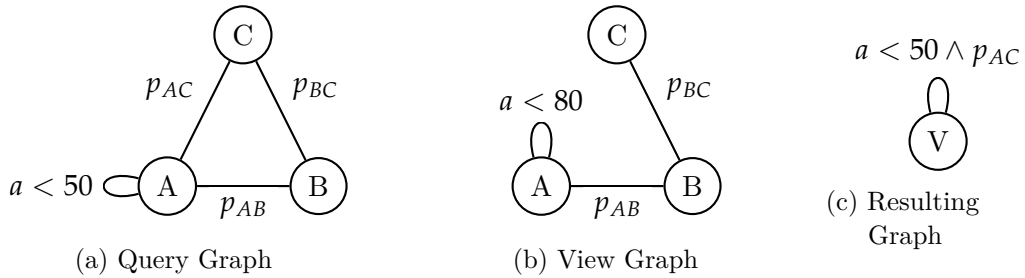


Figure 3.5.: Query Graph that contains more Edges and other predicates wrt. the View Graph

3.3.1. Definition of Matching Criteria

Whereas we previously checked whether the subgraph and view graph are completely equal, we will now work with a notion of “the output of the subgraph is computable from the output of the view”. We define $\text{output}(G)$ as the output relation of any query tree that can be re-built from G . Next, we need to define when the output of one graph is computable from another graphs output. To this end, let us first analyze which operations can be performed on top of a view, since these are the same operations that could be applied on top of the view graph.

The operations that can easily be introduced on top of a view are selections, to filter tuples, and projections, to filter attributes. So, we can remove superfluous tuples and attributes but we cannot add additional. Also, we can not change the cardinality of tuples. Newly introduced selections on top of a view can be used to filter out superfluous tuples, but only if the attributes that are accessed by the predicate are not projected away by the view. We will further call the attribute of the newly introduced selection the residual predicate $p_{\bar{r}}$.

These considerations lead to the criteria from Goldstein and Larson (cf. Chapter 2) [16] with respect to partial view matching. To be able to compute the output of the subgraph from the output of the view, we require that:

1. The view contains all tuples of the subgraph, so $\text{output}(G_X) \subseteq \text{output}(G_V)$, otherwise introducing the view removes tuples that may not be restored.
2. The view contains all attributes that are required by the query, otherwise the rest of the query may no longer be evaluated or the output attributes are unavailable. Let $\text{attrs}(p)$ be the attributes that a predicate accesses, then the required attributes are $\bigcup_{e \in (E_Q \setminus E_V)} \text{attrs}(p(e))$ and $\mathcal{S}(\text{output}(G_Q))$.
3. The view contains all attributes that are required to filter out additional tuples. We filter out additional tuples with the residual predicate, so the required attributes are $\text{attrs}(p_{\bar{r}})$.
4. The view contains all tuples with the same duplication factor as the original query. In bag semantics we can count tuples with the $\#$ operator, where $B\#x$ counts the occurrence of x in B . So $\forall t \in \text{output}(G_Q) : \text{output}(G_Q)\#t = \text{output}(G_V)\#t$.

If all of the criteria hold, then we can find a residual predicate and a projection such that:

$$\Pi_{\mathcal{S}(\text{output}(G_X))} \sigma_{p_{\bar{r}}} \text{output}(G_V) = \text{output}(G_X) \quad (3.1)$$

We will prove this after we explain how to find the residual predicate. We will not consider the residual predicate since all tuples will be projected to the query output schema later

on, and carrying additional attributes does not affect the query semantics. The meaning of the equation is, that we can use G_V with an additional projection and selection to replace G_X . In the world of query trees this implies that we can find a join-ordering of G_X that has the same output as the view tree with a projection and selection on top. The rest of the argumentation follows Section 3.2.2.

We now show how to implement an algorithm that checks the 4 properties and decides whether a view matches. Afterwards, we discuss how to derive the residual predicate and finally prove our claim that the output of G_X and G_V with residuals is equal.

3.3.2. Cardinality Criterion

Let us first consider the cardinality criterion. The cardinality of a tuple changes based on predicates that decide whether a tuple is included in the output (0 or 1) and on the relation between different inputs. A cross product $T_1 \times T_2$ for example would multiply the cardinality of every tuple in T_1 by $|T_2|$, whereas an inner join between $T_1 \bowtie_p T_2$ modifies the cardinalities of tuples in T_1 based on p and the contents of T_2 . This means, that additional tables in the view might change the cardinalities.

There are, however, joins that will never change the cardinalities of tuples, so called cardinality-preserving joins. Such a join would match every tuple of T_1 with exactly one tuple of T_2 , meaning that $\Pi_{S(T_1)}(T_1 \bowtie T_2) = T_1$. This for example occurs in an equijoin between a non-null foreign key in T_1 referencing a unique key in T_2 . In this case, firstly, no tuple can be discarded from the T_1 , since every corresponding key must exist in T_1 . Secondly, the cardinality of every tuple in T_1 can not change since there is exactly one matching tuple due to the uniqueness constraint in T_1 . So, extending the criterion, we could require that $V_V \setminus V_Q$ are only inputs which were connected by cardinality-preserving joins. We define $\text{Hub}(G_V) = (V_H, E_H)$ as the view graph that is constructed by removing all cardinality-preserving joins and their associated inputs from G_V . This changes the induced subgraph G_X from induced by V_V to induced by $(V_V \cap V_Q) \cup V_H$. These are all shared vertices that the query shares with the view, plus at least the vertices of $\text{Hub}(E_H)$. We will not work with query hubs in the remainder of this work, for details refer to [16].

3.3.3. Tuple Inclusion Criterion

The next observation is, that selections or joins which exists in the view but not in the query could potentially remove tuples from the output or again change the cardinalities. In Figure 3.5 we would have to check if the edge $\{(A)\} \in E_V$ with predicate $a < 80$ filters out any tuple that is contained in the query output. We can see that the query contains a similar edge $\{(A)\} \in E_Q$ with $a < 50$, which is more restrictive, so the predicate in the

view has to be satisfied for every tuple that also satisfies the query. Let us generalize this formally:

$$\begin{aligned}
\text{output}(G_X) \subseteq \text{output}(G_V) &\equiv \\
&\equiv \neg \exists t : t \in \text{output}(G_X) \wedge t \notin \text{output}(G_V) \\
&\equiv \forall t : \neg(t \in \text{output}(G_X)) \vee t \in \text{output}(G_V) \\
&\equiv \forall t : (t \in \text{output}(G_X)) \Rightarrow (t \in \text{output}(G_V))
\end{aligned}$$

Since $\text{output}(G_X)$ is defined by all of its predicates (see Appendix A):

$$\equiv \forall t : (\forall p_x \in \{p(e_x) \mid e_x \in E_X\} : p_x(t)) \Rightarrow (\forall p_v \in \{p(e_v) \mid e_v \in E_V\} : p_v(t))$$

We extend the definition of p to $p(E) = \{p(e) \mid e \in E\}$:

$$\equiv \forall t : (\forall p_x \in p(E_X) : p_x(t)) \Rightarrow (\forall p_v \in p(E_V) : p_v(t))$$

All predicates have to hold if their conjunction holds:

$$\begin{aligned}
&\equiv \forall t : \bigwedge (p(E_X))(t) \Rightarrow \bigwedge (p(E_V))(t) \\
&\equiv \bigwedge (p(E_X)) \Rightarrow \bigwedge (p(E_V))
\end{aligned}$$

This check is very time consuming, for example we could check if $\neg(\bigwedge (p(E_X)) \Rightarrow \bigwedge (p(E_V)))$ is satisfiable by employing a SAT solver. If it is unsatisfiable, we know that the view contains all tuples of the query. However, we can also conduct the check individually per view predicate, since

$$\bigwedge (p(E_X)) \Rightarrow \bigwedge (p(E_V)) \equiv \forall p_v \in p(E_V) : \bigwedge (p(E_X)) \Rightarrow p_v$$

If we trade accuracy (i.e. rejecting matching views, thus allowing false negatives) for speed, we do not need to check the conjunction of all query predicates and can instead continue to require that $E_V \subseteq E_Q$. So, every edge e_v will have a corresponding edge e_q that accesses the same relations. Then, we can check whether $p(e_q) \Rightarrow p(e_v)$. Finding the sweet spot between accuracy and complexity and associated effort remains an open research question.

There is one last optimization that we can apply if we only want to consider pairs of matching edges. Consider a query that contains the joins $R \bowtie_{r=t} T \bowtie_{t=s} S$ and a view with $R \bowtie_{r=s} S$. Then the edge $e_v = (r, s)$ from the view would not be contained in the query and the view is rejected. However, we can see that $r = t \wedge t = s$, so $r = s$ holds. To circumvent this, we can insert an additional edge (r, s) with predicate $r = s$ into the query graph, then $e_v \in E_Q$. That is also beneficial for join ordering, since we can now build join-trees where (R, S) is joined before (R, T) or (S, T) . In fact, some algorithms

to create a query graph already include a step where these transitive equality joins are inserted. In the worst case, all relations share equivalent attributes, so the graph will be fully connected. This takes $(k-1) + (k-2) + \dots + 1 \in \mathcal{O}(k^2)$ edges. However the size of equivalence classes is quite limited, and a maximum threshold k up to which edge insertion is performed can be defined. If the construction algorithm for query graphs does not insert the edges, then we can insert them when computing the equivalence classes that we will require in the next section.

3.3.4. Attribute Inclusion Criteria

There are three types of attributes that need to be included in the views output, defined in criteria 2 and 3: Attributes that are required to evaluate the rest of the query, attributes that are required to compute the output of the query and attributes that are required by the residual predicate. We call the set of required attributes RA and check the following.

$$RA = \bigcup_{e \in (E_Q \setminus E_X)} \text{attrs}(p(e)) \cup \mathcal{S}(\text{output}(G_Q)) \cup \text{attrs}(p_r) \subseteq \mathcal{S}(\text{output}(G_V)) \quad (3.2)$$

The rationale for checking only attributes of edges in $E_Q \setminus E_X$ in the first term is, that for edges in E_X which can be included in E_V , their predicate is already evaluated inside the view. If the edge is included, but not restrictive enough (see the example loop from Figure 3.5), then the predicate will contribute to the residual predicate. This also holds for edges which are not included in the view at all. The attributes of the residual predicate are checked in the third term.

Ensuring 3.2 is straightforward from its definition. However, we can increase the amount of matching views, if we allow renamings of attributes. So if a query requires b as output attribute from a view, but the views schema is $\mathcal{S}(\text{output}(G_V)) = a$, we could look for a predicate $a = b$. That would allow us to rename a to b and apply the view. Obviously, there could be equivalences $a = c$ and $c = b$, so we have to think about transitive checks as well.

Generalizing this gives the equivalence classes problem. We sketch a simple union-find approach to form equivalence classes, just like [16]. Start with an one elemented equivalence class $\mathcal{EQ}(a) = a$ for every attribute a . For every edge in $e_v \in E_V$, if $p(e_v) \equiv i = j$ where i and j are both attributes, unite $\mathcal{EQ}(i)$ and $\mathcal{EQ}(j)$. Also, if we still need to extend the query graph from the last step, insert the edge $(r(i), r(j))$ where $r(a)$ is the relation that an attribute comes from.

Then check whether for all required attribute an equivalent attribute is included in the output: $\forall a_q \in RA : \mathcal{S}(\text{output}(G_V)) : (\mathcal{EQ}(a_q) \cap \mathcal{S}(\text{output}(G_V))) \neq \emptyset$.

3.3.5. Finding the Residual Predicate

So far, the view matching algorithm assures that the view contains all tuples from the query with correct duplication factor and all required attributes. However, there could still be superfluous tuples left if the view has less restrictive predicates than the induced subgraph. Thus, before we can replace the original query graph with the view, we have to filter out the additional tuples. This is performed by introduction of a residual predicate that gets applied on top of the view.

In the simplest case, the residual predicate could be set to $p(E_X)$, so just a re-application of all query predicates that were included in the induced subgraph. If we further take the views predicates into account, we could use $p(E_X \setminus' E_V)$, where the $\setminus'$ operation subtracts only the edges $e_v \in E_V$ from E_X , where $e_q \in E_X$ such that $p(e_v) = p(e_q)$. For the most sophisticated approach, the residual predicate $p_{\bar{r}}$ can be reduced to the solution of the equation $\bigwedge(p(E_X \setminus E_V)) \wedge p(E_V) \wedge p_{\bar{r}} \equiv \bigwedge(p(E_X))$. Like the predicate implication, this is subject to ongoing research.

3.3.6. Full Algorithm

With all the pieces together, we can define the full view matching algorithm for SPJ views that checks for applicability wrt. to our defined criteria. Listing 5 shows an algorithm that works on the extended query graphs with equivalence edges.

Let us again discuss the scalability of this algorithm in terms of runtime complexity. We consider the main points that we discussed above, cardinality checks, tuple inclusion checks and attribute inclusion checks, aswell as replacing the induced subgraph with the view.

As discussed, set inclusion for the cardinality check is $\mathcal{O}(|V_V|)$. Generating the induced subgraph is close to $\mathcal{O}(|E_Q|)$ as discussed in Section 3.2.2. Assuming that predicate implication takes constant time, the tuple inclusion checks complexity is $\mathcal{O}(|E_V|)$. Building the residual predicate could be easy by just taking $|E_X|$ or computing a more complex predicate. We stick to the basic version which is essentially a no-op, $\mathcal{O}(1)$. Building and checking the equivalence classes is a union-find problem. We require means a series of $|E_V|$ union operations for building the classes and $|\text{attrs}(V_Q)|$ find operations, which is overall computable in $\mathcal{O}((|E_V| + |\text{attrs}(V_Q)| + |\text{attrs}(V_V)|) \cdot \alpha(|\text{attrs}(V_V)|))$, where $\alpha(x)$ is the inverse Ackermann function [19]. Let the maximum supported attributes per relation be A , so we can estimate that $|\text{attrs}(V_V)| \leq A \cdot |V_V|$. A constant factor can be dropped from big-O notation, so we end up with $\mathcal{O}((|E_V| + |V_Q| + |V_V|) \cdot \alpha(|V_V|))$ to perform the output availability check. Building the remap table is linear in the size of attributes that require remapping, which is again $\mathcal{O}(|V_Q|)$. Inserting the rename and selection operations on top of the view takes constant time $\mathcal{O}(1)$. Lastly inserting the view into the query graph and remapping all edges was already discussed in Section 3.2.3 and

shown to be $\mathcal{O}(|V_V| + |E_Q|)$. In total, this sums up to $\mathcal{O}(|V_V|) + \mathcal{O}(|E_Q|) + \mathcal{O}(|E_V|) + \mathcal{O}(|V_Q|) + \mathcal{O}(|V_V| + |E_Q|) + \mathcal{O}((|E_V| + |V_Q| + |V_V|) \cdot \alpha(|V_V|))$. We know that $|V_V| \leq |V_Q|$ and $|E_V| \leq |E_Q|$ respectively, otherwise the subset criteria would not hold, so we can reduce this term to $\mathcal{O}((|E_Q| + |V_Q|) \cdot \alpha(|V_V|))$. The inverse Ackermann function can be regarded as a constant for any reasonable input size, so we end up with $\mathcal{O}((|E_Q| + |V_Q|))$, linear complexity in query size.

```

1  Graph matchAndInsertView(Graph  $G_Q$ , View  $View$ ) {
2       $G_V := View.graph\_representation$ ;
3
4      for  $v_v$  in  $V_V$  {
5          if ! $V_Q.contains(v_v)$  { return  $G_Q$ ; }
6      }
7
8       $G_X := G_Q.get\_induced\_subgraph(V_V)$ ;
9
10     for  $e_v$  in  $E_V$  {
11          $e_q := G_X.find(e_v)$ ;
12         if ! $e_q.expression.implies(e_v.expression)$  {
13             return  $G_Q$ ;
14         }
15     }
16      $p_{\tilde{r}} := build\_residual\_predicate(E_X, E_V)$ ;
17      $remap := Remap()$ ;
18
19      $\mathcal{EQ} := build\_equality\_classes(E_V)$ ;
20
21     for  $a$  in ( $G_Q.output\_attributes$ 
22          $\cup E_Q.without(E_X).attributes$ 
23          $\cup p_{\tilde{r}}.attributes$ )
24     {
25         if  $\mathcal{EQ}(a) \neq \emptyset$  {
26              $remap.insert(\mathcal{EQ}(a).take\_any(), a)$ ;
27         } else { return  $G_Q$ ; }
28     }
29
30      $new\_vertex := Selection(p_{\tilde{r}}, RenameAttributes(remap, View))$ ;
31
32     return  $G_Q.replace(G_X, new\_vertex)$ ;
33 }

```

Listing 5: View matching on Query Graphs utilizing Residual Predicates

3.4. Outer Joins (SPOJ)

Up until this point, we restricted the query graph to inner joins and extended it by selections. However, outer joins can also be included in a query graph with hyperedges, as shown in Section 2.4.2. Thus, we would like to extend our approach to outer joins. We could start by checking the inclusion of hyperedges from the view inside the query and vice versa, since the hyperedges limit the join re-orderability and give a notion whether the corresponding query trees are still equal.

If we use the example in Figure 3.6 we can however show, that the residual predicate generation for SPJ queries no longer works. The query is $A \bowtie_{a=c} (\sigma_{b < 5} B \bowtie_{b=c} C)$ and the view is equal to it except for the selection predicate, which instead is $\sigma_{b < 10}$. In this example, the query selects $b < 5$ and the view selects $b < 10$ which is obviously less restrictive, so the predicate implication check passes. However, the result of this selection is used as the right side of a left outer join. For example if we use $A = \{(1_a, 7_a)\}, B = \{(1_b), (7_b)\}, C = \{(1_c), (3_c), (7_c)\}$, then the selection and inner join evaluates to $\{(1_b, 1_c)\}$ in the query and $\{(1_b, 1_c), (7_b), (7_c)\}$ in the view. So far we could still eliminate the additional tuple, but after applying $A \bowtie_{a=c}$ to the result, we have $\{(1_a, 1_b, 1_c), (7_a, \text{null}_b, \text{null}_c)\}$ in the query and $\{(1_a, 1_b, 1_c), (7_a, 7_b, 7_c)\}$ in the view. So the tuple for 7_a is wrong in the view, because it contains values for b and c instead of nulls.

Still we can see that the view does contain more information than the query. We just have to find a way to replace tuple values with nulls. This process will be called nullification.

3.4.1. Nullification

One idea would be to introduce a new operator that will access tuples matching a given predicate and set certain attributes to null. The same can be achieved with an outer join, projection and renaming. Let V' be a copy of V with attributes a', b', c' , then we can use

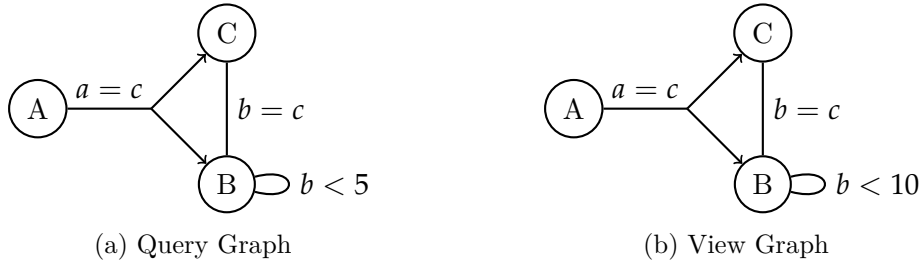


Figure 3.6.: Outer Joins with Predicate Implication

a self outer join:

$$\begin{aligned} V \bowtie_{a=a' \sigma_{b' < 5}} V' &= \{(1_a, 1_b, 1_c), (7_a, 7_b, 7_c)\} \bowtie_{a=a' \sigma_{p_{b'} < 5}} \{(1_{a'}, 1_{b'}, 1_{c'}), (7_a, 7_{b'}, 7_{c'})\} \\ &= \{(1_a, 1_b, 1_c), (7_a, 7_b, 7_c)\} \bowtie_{a=a'} \{(1_{a'}, 1_{b'}, 1_{c'})\} \\ &= \{(1_a, 1_b, 1_c, 1_{a'}, 1_{b'}, 1_{c'}), (7_a, 7_b, 7_c, \text{null}_{a'}, \text{null}_{b'}, \text{null}_{c'})\} \end{aligned}$$

Now both tuples of the Query are available and it remains to filter out additional predicates:

$$\begin{aligned} \Pi_{(S' \cup \{a\}) \setminus a'} \{(1_a, 1_b, 1_c, 1_{a'}, 1_{b'}, 1_{c'}), (7_a, 7_b, 7_c, \text{null}_{a'}, \text{null}_{b'}, \text{null}_{c'})\} \\ = \{(1_a, 1_{b'}, 1_{c'}), (7_a, \text{null}_{b'}, \text{null}_{c'})\} \end{aligned}$$

With a renaming of b', c' to b, c this is equal to the queries output. Also remember that we are using bag semantics, so Π is not eliminating duplicates.

However, this approach is not always feasible, as there could be cases where a less restricting predicate in the view removes tuples, that cannot be restored from the views output. Consider a query $\sigma_{b=\text{null}} (A \bowtie_{a=b} \sigma_p B)$, which resembles an anti-join. Every tuple in B for which the predicate p evaluates to true could potentially find a join partner in A . And every tuple of $t \in A$, where $t_a \neq \text{null}$ that finds a join partner will not be included in the output, since $t_a = t_b \neq \text{null}$. This means that the less restrictive p is, the more tuples can find a join partner and thus will not be included in the output. So, it no longer suffices to check the implication of $p_v \Rightarrow p_q$.

Analyzing the case shows, that there is a predicate surrounding the outer join which is accessing tuples from B and is not null-rejecting¹. Thus, we require that either there is no predicate that accesses B outside of the outer join, or that if it exists, it is null rejecting.

Let us now sketch the general approach for nullification. So far, we used a selection predicate, which we will call the nullification predicate p_n , a join predicate p_j and sets for the projection $\mathcal{P}$ and rename. Additionally, we still require the overall residual predicate $p_{\bar{r}}$ that is used to filter out tuples from the output.

$$V_{\text{adapted}} = \Pi_{\mathcal{P}} (\sigma_{p_{\bar{r}}} V) \bowtie_{p_j \sigma_{p_n}} (\sigma_{p_{\bar{r}}} V') \quad (3.3)$$

In the following, we will discuss how to compute each of the predicates and projection and rename sets. All of this will be conducted on a view that has only a single left-outer join. We will later talk about how to extend it to a view with multiple outer joins in query graph representation. Also, we will discuss how to avoid the whole nullification process if possible.

¹ Note that using the join disjunctive normal form also fails for this case: $\sigma_{b=\text{null}} (A \bowtie_{a=b} \sigma_p B) \equiv \sigma_{b=\text{null}} (A \bowtie_{a=b} \sigma_p B \oplus A)$ At this point, would like to apply Equations (2.18) and (2.19) but the predicate $b = \text{null}$ is not only referencing A and also not strong.

Join Predicate

The join predicate needs to guarantee that each tuple finds exactly one or zero join partners. The join partner either is the tuple itself, if it is not filtered out by the nullification predicate, or there is no join partner. To guarantee that there are exactly one or join partners for the tuple, we need to use an equality predicate over a candidate key (a set of attributes that is unique for each tuple) of the tuples. Otherwise a tuple could find multiple join partners and its cardinality would change, thus violating the view matching criteria. If the view has a unique, or primary key, then it can be used here. If it does not have a unique key, then we can try to find a candidate key by looking at its inputs.

If the inputs are connected solely using cardinality-preserving joins, every tuple is uniquely identifiable by any of the base relations unique keys. If there is a join that does not preserve cardinality, we can build a composite key out of the keys for each input relation to the join.

So for each join $S \bowtie T$, with respective candidate keys K_S and K_T , we enforce that it is either cardinality-preserving, then the output relation has all candidate keys of its inputs $K_S \cup K_T$ (then K_S or K_T must be non-empty). Or the join is not cardinality-preserving, then both input relations must have candidate keys K_S and K_T respectively and the candidate key for the output of the join is $K_S \times K_T$. Since we require the key on top of the view, we only consider keys that are available in the output of the view.

Let K be a candidate key of the view. This key is a set of all the attributes that we require to identify a tuple, so can use $p_j = \bigwedge_{k \in K} k = k'$ where k' is k in V' as the join predicate in $V \bowtie_{p_j \sigma_{p_n}} V'$.

Projection Set

Let the outer join be $R \bowtie L$, where L and R can be any SPJ expressions. Then all attributes of R must potentially be nullified and all attributes of L must be kept as is.

Remember that we clone the view V to V' , and apply the nullification predicate to V' to filter out those tuples that should become null. We rename R in V' to R' . So, we need to keep the attributes in L of V and use the attributes of R' in V' . This is expressed by first projecting it on $\mathcal{S}(L) \cup \mathcal{S}(R')$ and then renaming R' back to R with a renaming operation. Note that some of the attributes of $\mathcal{S}(L)$ or $\mathcal{S}(R')$ could be missing in $\mathcal{S}(\text{output}(V))$. That however does not matter for the projection, since it just ignores the missing attributes.

Nullification Predicate

The nullification predicate identifies which tuples should contain nulls instead of values. Let the outer join be $L \bowtie_q R$, where L and R are again arbitrary SPJ expressions. Then, all tuples that potentially need to be nullified come from R . It is thus natural, that we need to consider all predicates in R , as well as q . We already discussed how to build a residual predicate in the previous chapter, and the same applies here. So we take all predicates in $R \cup q$ that do not have an equal predicate in the query and q if the outer join predicate in the query is different from q and build a residual predicate out of them. This residual predicate is then the nullification predicate.

Application on the Query Graph

So far we discussed how to theoretically extract all parts that we require for nullification of a single outer-join view. To be able to apply these mechanisms however, we also have to define how they work on a query graph. So far, we argued about outer-joins $L \bowtie_q R$ and then used L and R , so it remains to identify these SPJ components in the graph.

Let us start by inspecting the query graph for a view that contains a single outer join and no cross products. This join will form a (hyper-)edge in the view graph, with the associated semantics that either side of the hyper-edge has to be evaluated before the outer join itself can be evaluated. In the rebuild procedure this meant, that the hyperedge can be picked only after both sides are unified into a single node. So we can build a similar procedure that will compute L and R , by keeping additional information in the rebuild, until the outer join is encountered. At the point where the outer join is picked, all inputs that can not be re-ordered to come after it must be built, so we immediately know R based on the additional information that each input to the outer join tracked.

We sketch a similar procedure to the rebuild algorithm now, which will compute p_j , p_n and the projection set in a single pass over the view graph G_V . The approach is based on union-find over G_V , and we do a case distinction over what kind of edge we are picking. We will procedurally build the composite key, projection set and nullification predicate, which requires us to store intermediate results for the composite key $KEYS(R)$ and nullification predicate $NPRED(R)$. $NPRED(R)$ will be a tuple that stores pairs of predicates from query and view to later assemble them into p_n . For base relations, $KEYS(R)$ will be the set of candidate keys and $NPRED(R) = (true, true)$.

- If the edge $e_v = (T_1, T_2)$ belongs represents an inner join or selection (T_1 and T_2 could be equal if e_v is a loop) with predicate p_v , we look that edge up in E_Q with predicate p_q and check for $p_q \Rightarrow p_v$. If e_q does not exist or the implication does not hold we can stop, according to the view matching procedure for SPJ views. We

then join R and S into a new vertex T like in the rebuild procedure. We unite the attributes that we tracked as follows: Also, we compute the candidate key.

$$\begin{aligned} KEYS(T) &= \begin{cases} KEYS(T_1) \cup KEYS(T_2) & \text{if } T_1 = T_2 \text{ or } T_1 \bowtie_{p_v} T_2 \text{ is cardinality preserving} \\ KEYS(T_1) \times KEYS(T_2) & \text{otherwise} \end{cases} \\ NPRED(T) &= \begin{cases} NPRED(T_1) \wedge_t NPRED(T_2) & \text{if } p_q = p_v \\ NPRED(T_1) \wedge_t NPRED(T_2) \wedge_t (p_v, p_q) & \text{otherwise} \end{cases} \end{aligned}$$

Where we define $(a, b) \wedge_t (c, d) = (a \wedge c, b \wedge d)$ as the lift of conjunction to tuples.

- If the edge belongs to an outer join $T_1 \bowtie_q T_2$, then we identified $R = T_2$. We again assert that the query also contains an outer join with this predicate, or a less restrictive predicate $q_q \Rightarrow q_v$. Thus, we know that the residual predicate must consider the predicates from $NPRED(T_2)$ (plus the pair (q_q, q_v) if $q_q \neq q_v$). In the most simple case, just use the second entry of $NPRED(T_2)$ plus q_q , which is the conjunction of all query predicates plus the predicate of the outer join. More sophisticated ideas to build p_n can be considered as according to Section 3.3.5. Also we know $\mathcal{S}(R)$, which we require for the projection set, since T_2 has to be completed. T_1 could still be extended by other joins that are applied after the outer join. The candidate key is still not finished, so we continue to build the new vertex T with building $KEYS(T)$ analogous to the previous case for selections or inner joins. $NPRED(T)$ will also have to be set, to compute the overall residual predicate. We will thus set it to $NPRED(L)$, since all predicates from R are used for nullification of tuples and not for general exclusion.

When the algorithm is finished, we are left with a single node T . We have to check whether any of the candidate keys is available in the views output $\exists K \in T : k \subseteq \mathcal{S}(\text{output}(G_V))$. If none is, we cannot build the join predicate p_j and have to abort. We also already computed p_n when we found the outer join, so it remains to build the projection set. Since we know $\mathcal{S}(R)$, we can build the part of $\mathcal{S}(L)$ that is included in the views output attributes as $\mathcal{S}(L) \cap \mathcal{S}(\text{output}(G_V)) = \mathcal{S}(\text{output}(G_V)) \setminus \mathcal{S}(R)$. This holds, because every attribute in the output has to come from either L or R .

We now have everything we need for nullification: p_j can be built from a candidate key, p_n was computed when the outer join occurred, and the projection set wrt. to the views output can be built.

What is left to build is the residual predicate $p_{\bar{r}}$, which can be computed from $NPRED(T)$, according to Section 3.3.5. At last, we can insert the view as for SPJ queries with the extensions given in Equation (3.3).

The whole run-time complexity for the approach is the complexity for regular view matching plus a union-find on the query graph, which takes E_V union operations on

V_V starting nodes, so $\mathcal{O}((|E_V| + |V_V|) \cdot \alpha(|V_V|))$. The set of candidate keys could grow exponentially in $O(k^{E_V})$ if no join is cardinality preserving. In an implementation, we can reduce that complexity by not checking for each entry of $KEYS(T_1) \times KEYS(T_2)$, whether it is included in the output schema, to significantly reduce intermediate result size, or even better giving the materialized view a unique key.

To extend the approach to multiple outer joins, we need to remember p_n and $\mathcal{S}(R)$ for every outer join $L \bowtie R$. Then we can repeatedly apply Section 3.3.5. First for the outer join which will be evaluated first in the view-representation of the query graph and then traversing upwards.

Avoiding Nullification

There is an exception where we can have outer joins and do not require nullification, which is if the attributes that would require nullification are never required in the query, or if there is a null-rejecting predicate right on top of the outer join.

Another idea to avoid nullification is to go back from predicate implication to predicate equality, as we first did for SPJ queries. We do not even need to enforce predicate equality for every predicate, but just for the ones that are part of an outer join $L \bowtie_q R$, so all predicates in R and q . While this saves time and complexity, it also increases the amount of false negatives.

3.4.2. Join degradation

There is one more thing that we can discuss, which is join degradation. Until now, we enforced that if the view had an outerjoin, then the query requires a matching outerjoin. This however is not necessary, since we could eliminate all the null tuples with the residual predicate.

Suppose a query contains the join $A \bowtie_p B$, then this part can be replaced by a view $A \bowtie_p B$ if we introduce the residual predicate $p_{\bar{r}} = \bigvee_{b \in \mathcal{S}(B)} b \neq \text{null}$ and B does not contain a null-tuple. If B contained the null-tuple, then we would accidentally filter it from the output, even if it was there legitimately.

Implementing join degradation could decrease the false negative rate and make views more applicable, however we leave the details to future work.

3.5. Non-SPOJ Operators

We presented an algorithm that can match SPJ views, and we sketched an extension for outer joins. However, relational algebra contains much more operators, such as anti-joins, groups, or the set operations (union, intersection, difference, etc.), and view matching

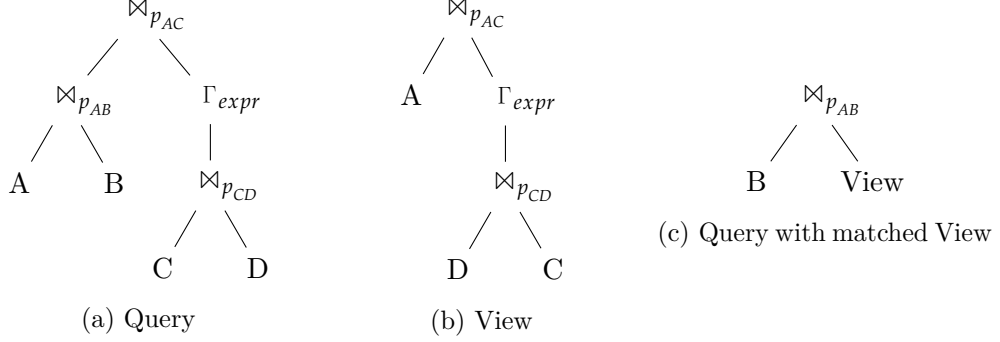


Figure 3.7.: Example of matching view with aggregation

should be able to match views and queries that contain them to some extent. We will call these operators non-SPOJ operators. Figure 3.7 shows the query trees for a query and view containing an aggregation. As we can see, the view matches the query, and could be inserted as shown on the right side, so we would like to extend our view matching algorithm to respect that.

The general constraints that aggregations impose on the join re-ordering problem are unclear and subject to ongoing research [20], and we do not know about an approach where aggregations are included in a query graph suitable for join ordering. The same holds for many other operators of relational algebra, but views containing them should at least be matched if it obvious to do so.

By obvious, we mean that the query tree below the non-SPOJ operator produces the same output in query and view, and that the non-SPOJ operator is equal. Join ordering of the subtrees is irrelevant, but all predicates have to be exactly equal, because we cannot apply a residual predicate between the subtree and the non-SPOJ operator. In general we also do not know what the non-SPOJ operator does, so we also can not compute a more complex adaption of the view output as we sketched for outer joins. Overall this means a query with $R \circ_p S$, where $\circ$ is any non-SPOJ operator matches a view $R' \circ'_q S'$, if R' partially matches R according to the criteria of Section 3.3, $\circ = \circ'$, $p = q$ and S' is equivalent to S , as in Section 3.2.

We still want to align this matching with our graph-based approach, so we will show how non-SPOJ operators are usually handled for the join-ordering problem. We will then describe a way to extend the query graph with non-SPOJ operators and afterwards discuss how to perform view matching on the new query graph.

One last note is, that non-SPOJ operators could be handled more effectively, to further reduce false negatives. Outer joins for example are just a special case for connecting operators that can be represented in a query graph and also handled more efficiently with

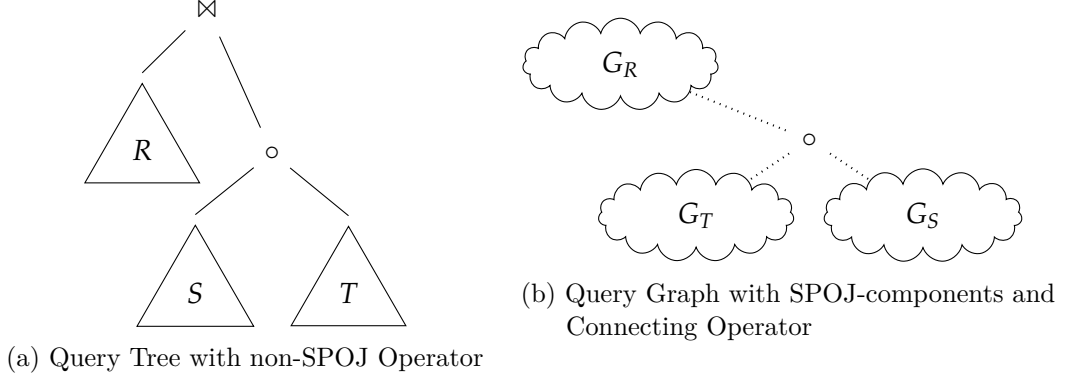


Figure 3.8.: Data Structures for non-SPOJ Query

the nullification approach, as sketched before. Also tuples from a union can potentially be filtered later on, or some aggregations can be computed from others, for example an AVG from SUM and COUNT. We will further discuss this in future work (Chapter 6).

3.5.1. Connecting Operators and SPOJ-Components

Remember, that the join ordering problem was finding a query tree that is optimal in terms of cost, where cost is defined based on the size of intermediate results. As noted, non-SPOJ operators in general can not be re-ordered easily, but we could re-order their subtrees and the tree in which they occur. Thus, the join-ordering algorithm will be applied to each of the subtrees of a non-SPOJ operator, and then find the optimal evaluation plan of the operators that are re-orderable above of the non-SPOJ operator.

If we take the query tree in Figure 3.8 as an example and assume that the subtrees R , S and T do only contain SPOJ operators, then join-ordering will be applied to each of the subtrees individually. Afterwards, they are being put back together where S and T still kept as children of $\circ$. The position of $\circ$ in the join tree R could change, because to R , $\circ$ looks just like a base relation.

If we apply join ordering to S and T that means, that the join ordering algorithm will first build a query graph for each and then find the optimal evaluation plan respectively. For view matching this means, that the graphs of all three, S , T and R are being built and come for free. So we can build a big graph, in which we connect R , S and T , remembering $\circ$ as the connecting operator between each individual graph as shown in fig. 3.8 on the right side. We will call the individual graphs for the subtrees SPOJ-components, represented as clouds, to differentiate between the overall query graph and the subgraphs. Now, we can perform view matching on queries graphs consisting of SPOJ-components and connecting operators.

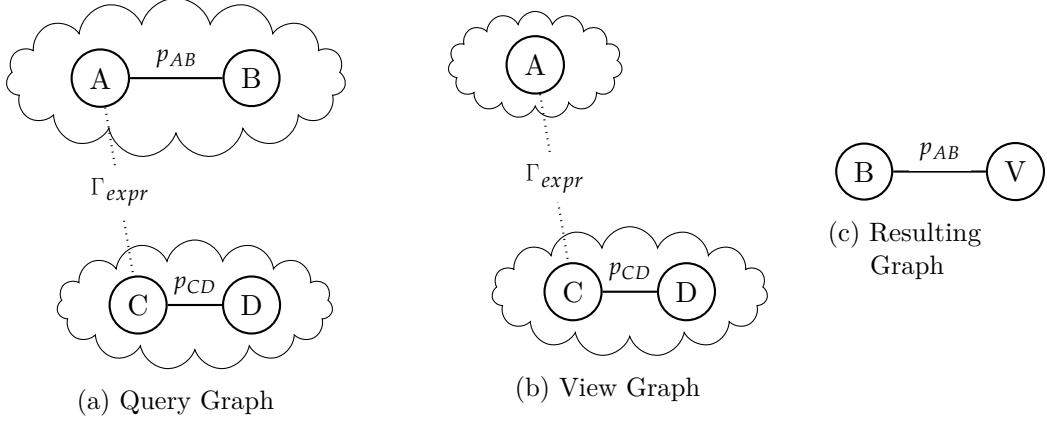


Figure 3.9.: View matching with Connecting Operators and Subgraphs

3.5.2. View Matching with Connecting Operators

The first thing to note is, that a graph consisting of a single SPOJ-component is exactly what the view matching algorithm handled previously. So, matching SPOJ queries works just as before. Further this means, that we can match an SPOJ component, that has non-SPOJ operators below it, regularly if we regard the whole non-SPOJ subtree as a single vertex. Additionally, we still have to check that these non-SPOJ operators produce the same output in query and view. Since they are still allowed to differ in join-ordering, we can apply our first view matching algorithm on graphs to each of their respective inputs as SPOJ-components. Technically we defined the first algorithm without outer joins, but we can add outer joins by just checking the equality for the hyperedge, since we avoid nullification by checking all predicates for equality as described in Section 3.4.1.

In Figure 3.7 the SPOJ-component below the non-SPOJ operator is $C \bowtie_{p_{CD}} D$ or $D \bowtie_{p_{CD}} C$ respectively. We transform the query tree into graph representation to obtain Figure 3.9, where we can partially match the $\{A, B\}$ component with $\{A\}$ from the view, and see that the $\{C, D\}$ component and non-SPOJ operator Γ_{expr} are equivalent. So, the view matches, and we can insert it into the top SPOJ component as usual.

In our example, the query had just a single non-SPOJ operator, so we will now generalize the matching. If we have multiple non-SPOJ operators, the resulting graph will be a tree of SPOJ components. The intuition behind this is, that the query tree will be cut in pieces at every non-SPOJ operator and each piece will form a SPOJ component. So, we have the two graphs for view and query which both form a tree of SPOJ components. The view matching algorithm could now traverse the query's SPOJ-tree and consider two cases:

- In the first case, the view matches at the current position in the SPOJ-tree. Then

for every connecting operator in the view, equivalence of the operator and all of its subgraphs has to be ensured.

- In the second case, the view does not match at the current position, so the algorithm recurses into each child SPOJ-component in the query and checks matching of the child and the view.

The approach would take $\mathcal{O}(|K|) \cdot \mathcal{O}(\text{matching})$ time-complexity where $|K|$ is the amount of SPOJ component in the query graph. To avoid this traversal of the tree, we can give each SPOJ-component a unique identifier. A bottom-up matching approach could thus try to identify the position of all the views SPOJ-components in the query graph, and if the positions share a common parent, the views root component has to partially match this parent.

3.6. Choice of matching view

There is one last point that we did not addressed yet, which is choosing whether a matching view should be used. If we have a query graph and multiple views would match the graph, it should be decided if one or more views should be inserted and if so which. While this is mostly a cost-based decision that the optimizer needs to make, we give a few insights into this problem as well. As a motivation, consider the query $A \bowtie_p B \bowtie_q C$. If we have a view that $V_1 = A \bowtie_p B$ and another view $V_2 = A \bowtie_{p'} B \bowtie_{q'} C$ where p' is less restrictive than p and q' less then q . Would it be better to use V_1 and compute the join $V_1 \bowtie_q C$, or to compute the selection $\sigma_{p'}(V_2)$? We will answer this question in general by first extending the cost function for evaluating query trees with respect to views and then showing how to integrate view matching into the optimizer.

3.6.1. Cost Function for Views

We previously defined the cost function $\mathcal{C}(T)$ for a query tree in Equation (2.24). Back then, we did not consider the cost to evaluate predicates, so let us now extend the definition by $\mathcal{C}(\sigma_p(T)) = |T|_e + \mathcal{C}(T)$. Then the cost for inserting a view is $\mathcal{C}(\sigma_{p'} V) = |V|_e + \mathcal{C}(V) = |V|_e$, since V can be seen as a base relation.

We define the cost for evaluating the query without the view as $\mathcal{C}_G(G_Q) = \mathcal{C}(T_Q)$, where T_Q is the optimal query tree for G_Q . The problem is, that the view is not necessarily a subtree in the optimal query tree of G_Q . So, the cost of the query with inserted view can only be computed on the query graph with inserted view (G_I).

We can now compare $\mathcal{C}_G(G_Q)$ with $\mathcal{C}_G(G_I) + |V|_e$ and choose the cheaper option. However, this approach does not scale to multiple views. If we have two matching views V_1 and V_2 , we would need to check the costs without a view, with V_1 , with V_2 and with

both views. In general we would need to check 2^n costs, when we have n views, plus the overhead to check whether views overlap and cannot be applied at the same time.

3.6.2. Integrating View Matching into the Optimizer

To avoid these considerations, we can directly integrate the choice whether to use a view into the optimizer that chooses the join re-ordering. A common solution to join ordering is dynamic programming [8]. The idea is to find the optimal overall plan by finding the best join trees for subgraphs, as in Listing 6. The algorithm `DPsize` iteratively builds the

```

1  DPsize ( $R = \{R_0, \dots, R_{n-1}\}$ )
2  for  $\forall R_i \in R : \text{dpTable}[\{R_i\}] = R_i$ 
3  for  $\forall 1 < s \leq n$  ascending :
4    for  $\forall 1 < s_1 \leq s$  :
5      for  $\forall S_1 \subset R : |S_1| = s_1, S_2 \subset R : |S_2| = s - s_1$ 
6        if  $S_1 \cap S_2 \neq \emptyset$  continue
7        if  $\neg(S_1 \text{ connected to } S_2)$  continue
8         $p = \text{dpTable}[S_1] \bowtie \text{dpTable}[S_2]$ 
9        if  $\mathcal{C}(p) < \mathcal{C}(\text{dpTable}[S_1 \cup S_2])$ 
10           $\text{dpTable}[S_1 \cup S_2] = p$ 
11  return  $\text{dpTable}[\{R_0, \dots, R_{n-1}\}]$ 

```

Listing 6: Pseudocode for a DP-based optimizer

best plan through dynamic programming. A plan of size 1 is just the base table, and a plan of size s consists of a plan s_1 and a plan $s - s_1$. So the best plan for size s is the best possible combination of smaller plans, which are already optimal by design of DP. To integrate view matching, we could first run the view matching algorithm for all views V_i and remember which base tables they replace, by checking the overlaid subgraph. Let the set of replaced tables for V_i be X_i , then we can set $\text{dpTable}[X_i] = V_i$. That is, we precompute a plan for the relations that the view could replace, which is exactly using the view and its residual predicate. In line 9, where the algorithm checks whether the current plan for $S_1 \cup S_2$ is better than the stored plan, it will compare any plan that could be better than the view, with the view. For the costs of the view, we use our previous definition of $\mathcal{C}(V_i) = |V_i|_e$. So we preserve the optimality criterion of the DP algorithm, and the optimizer can choose which views it should integrate.

Thus, we can integrate the full view matching approach on query graphs into the DP based optimizer.

4. Implementation

We implemented view matching as graph overlay problem in the academic database system umbra. It is important to note, that umbra currently does not support materialized views and instead stores regular views as query strings. We will talk about the impact of this on our implementation whenever it affects our design decisions.

In the following, we will discuss the details of our implementation that allow us to efficiently execute the view matching algorithm. We will first describe the data structures that we use to efficiently compute set inclusions and equality checks. Next, we show how the fine tuning in the precision and complexity of predicate matching can be separated from the view matching implementation. In the end, we discuss the shortcomings of our implementation and what is missing in comparison to the algorithm we described in the previous chapter.

4.1. Data Structures

The designed view matching algorithm requires an efficient way to compute set inclusions for vertices and edges, as well as lookups of corresponding edges from one graph in another. We identify vertices with a unique identifier and edges by sets of vertices. Thus, we can perform vertex and edge lookups in $\mathcal{O}(1)$ and respective set inclusion and equality in $\mathcal{O}(n)$ as we will show in the following. Furthermore, we discuss how SPOJ-components can be identified to improve the performance of non-SPOJ view matching.

4.1.1. Unique Identifiers

We use integer ids to identify each vertex in the query graph. Remember that vertices correspond to input relations, so we can assign each input a unique id during query compilation. To ensure, that the input ids in query graph and view graph are the same, we build a mapping from input (for example the qualified name for a base relation) to id and then iterate the prebuilt view graph to assign each input that is available in the query the id from the mapping. All other inputs will be assigned ids that are outside of the id-range of the query. The whole procedure is in $\mathcal{O}(|V_V|)$ since a lookup has to be performed for every vertex of the view.

Additionally we identify edges by sets of vertices that they connect, so we need to assign the correct set to any edge. This takes another $\mathcal{O}(|E_V| \cdot k)$, where k is the amount of vertices that an edge references. This is limited in practice, so we end up with $\mathcal{O}(|V_V| + |E_V|)$ to build the unique identifiers.

Our implementation uses bitsets as representation of sets of integer ids. Therefore we can represent all vertices in a graph as a single integer $B(V) = \sum_{i=1}^n 2^{\text{id}(v_i)}$, where $B(V)$ represents the bitset of V . We can do something similar for an edge $e = (R, S)$, because R and S are sets of nodes, so $B(e) = (B(R), B(S))$.

4.1.2. Set Inclusion and Equality

We can now check whether $V_V \subseteq V_Q$, by first testing whether $|V_V| < |V_Q|$. If this is not the case, we can already stop. Otherwise, we continue testing $B(V_V) \& B(V_Q) = B(V_Q)$ where $\&$ is the bitwise AND of two integers. If we want to work with query hubs, then we would first need to reduce V_V to $\text{Hub}(V_V)$ and check the inclusion $\text{Hub}(V_V) \subseteq V_Q$ exactly as before. To check set equality we could check $V_Q \subseteq V_V$ respectively, but it is more efficient to just check whether $|V_V| = |V_Q|$. If this holds in addition to inclusion $V_V \subseteq V_Q$, then V_V and V_Q must be equal. Subset inclusion and equality for vertex sets is $\mathcal{O}(V_Q)$, since the bitwise AND of two arbitrarily large bitsets takes linearly increasing CPU instructions processing 64 Bit each (or with SSE extensions up to 128 Bit). The scaling is thus quite low with $\frac{|V_Q|}{64}$.

We also have to check whether $E_V \subseteq E_Q$, while identifying each corresponding edge e_q for an edge e_v to be able to check their predicates later on. To this end, we build a hashmap $\text{Edgemap} = (\text{Bitset}, \text{Bitset}) \rightarrow \text{Edge}$ for the query graph. Programming languages usually do not support unordered tuples, so we have to improvise by checking both edge directions, when the edge is undirected (e.g. for inner joins). The following code snippet shows how to look up a corresponding edge.

```

1  (R, S) := e_v
2  e_q = Edgemap[(R, S)]
3  if e_q == null && e_v.is_reversible {
4      e_q = Edgemap[(S, R)]
5  }
```

If e_q is still empty afterwards there was no corresponding edge for e_v and we can terminate the view matching algorithm. This check has to be performed for every edge in E_V and lookup in a hashmap is $\mathcal{O}(1)$. To build the hashmap, we have to insert all edges from E_Q , which is also in $\mathcal{O}(1)$ for each insert. So in total checking edge inclusion (or equality) is in linear in size of query and view $\mathcal{O}(|E_Q| + |E_V|) = \mathcal{O}(|E_Q|)$.

4.1.3. Connecting Operators

We previously discussed that SPOJ components could offer a unique identifier, to speed up non-SPOJ query matching. In our implementation, we do not compute a unique identifier for each SPOJ component, but we associate each with the bitset of its vertices. We can then build a map from vertex-set to SPOJ component.

A map can only be used to lookup equivalent items, so in this case the vertex set must match. That makes it unapplicable for the views root SPOJ component, since this component could have less vertices than its corresponding query component. However, we can quickly check whether subcomponents of the view are contained in the query, because we enforced their equivalence.

4.2. Predicate Matching

We discussed how the inclusion $E_V \subseteq E_Q$ can be implemented and how a corresponding edge is found, but we did not yet discuss predicate matching. In Section 3.3.3, we discussed that $\bigwedge(p(E_X)) \Rightarrow p(e_v)$ should hold for every edge $e_v \in E_V$. We showed that checking $p(e_q) \Rightarrow p(e_v)$, where e_q is the corresponding edge of e_v in the query can speed up the check but reduce its accuracy. Furthermore, we argued that the sweet spot between speed and accuracy is highly implementation and usecase dependent.

To this end, we design a black-box for predicate implication $\bigwedge(p(E_Q)) \Rightarrow p(e_v)$, the implication oracle. We supply $p(E_Q)$, $p(e_q)$ and $p(e_v)$ and the implication oracle outputs **true** if the implication holds. If the implication outputs **false** it does not necessarily mean that the implication is either wrong or too hard to prove for the oracle.

With this approach, we can implement different oracles which can be changed out depending on problem size or choice of a database administrator. Our sample oracle implementation checks the syntactic equivalence of e_q and e_v , because solving the implication problem was out of scope for this work.

4.3. View Insertion

When we determined that a view matches, we still need to insert it into the query graph. Therefore, we must find all vertices of the view that have to be replaced, remove them and insert a single vertex N for the view. Finding the vertices that have to be removed is straightforward by checking $v_q \in V_V$ for all $v_q \in V_Q$. For the edges $e_q \in E_Q$ we need to consider four cases.

1. The edge is contained in the view, $e_q \in E_V$. Then either $p(e_q) = p(e_v)$, or the predicate has to be considered for building the residual predicate. Currently we

assert equivalence, so the predicate will be true for any tuple from the view. We can just remove the edge.

2. The edge connects only vertices that will be replaced by the view, but is not included in the view. Then we remove the edge but insert a new self-loop on top of the new vertex, with predicate $p(e_q)$.
3. The edge connects vertices outside of the view with vertices inside it. Let the edge be $e_q = (R, S)$, then we check whether $R \cap V_V$ is nonempty. If it is, we update it $R' = (R \setminus V_V) \cup \{N\}$. We do the same for S and insert a new edge (R', S') that replaces e_q .
4. The edge does not intersect the view, so it can be left as is.

The first two cases do not require special attention. The set intersection $R \cap V_V$ can be checked by computing $B(R) \ \& \ B(V_V) > 0$. This is technically $\mathcal{O}(|V_V|)$ but practically scales with a factor of $1/64$ as discussed, so close to constant time for reasonable input sizes. The same is true for $(R \setminus V_V) \cup \{N\}$ which in bitsets is $B(R) - B(V_V) + id(N)$. Checking which of the four cases applies, requires us to check whether both R and S are included in V_V , and the process has to be applied to each edge in E_Q . Thus we have $\mathcal{O}(|E_Q| \cdot |V_V|)$, with a small factor for V_V for view insertion.

Since the database system does not support materialized views, we build a query tree out of the view graph and put a TempScan operator on top. This operator prevents that join ordering will be changed later on (and would invalidate view insertion) and can be easily replaced by a MaterializedViewScan, as soon as they are implemented.

4.4. Shortcomings

Our implementation falls short in several aspects that we will also discuss. One of the biggest lacks of our implementation is that we do not check for output availability (attribute inclusion criteria, Section 3.3.4). We instead only use views that have a **SELECT *** clause, so all outputs will be available for the query. This way we also do not have to implement union-find for equivalence classes.

We discussed in Section 3.3.3 that we can insert new edges for equivalent joins, such as $R \bowtie r = sS$ for $R \bowtie_{r=t} T \bowtie_{t=s} S$. This step is already implemented in the optimizer, to generate optimal join plans where in this case e.g. R is first joined with S .

In Section 3.4.1, we sketched an approach for nullification, when outer joins are present. Instead of implementing the nullification process, we assert that all edges below an outer join are present in both, query and view and have equal predicates. Along these lines we

also did not implement join degradation, where we could use an outer join in a view to substitute an inner join in the query by applying a null rejecting predicate on top.

We furthermore did not implement an algorithm to detect extension joins and generate the hub of the view. This extension could be added easily and would decrease the number of false rejections.

Lastly, due to some implementation constraints, we are computing the query graph for view matching separately from the query graph that is used by the optimizer. Thus, we have to build query graph and view graph and transform both into tree representation again. The optimizer will then build a new query graph for join ordering, which introduces additional overhead. Without changing this approach we could at least omit building the query tree for the view, if the database system would support materialized views, since we would not need a representation of the view that can be evaluated. Followingly, we also did not adapt the optimizer to be view-matching aware. Instead, we iterate over potentially matching views from biggest to lowest and try to match each. If a view matches, we immediately insert it into the query graph. This way, we cannot guarantee optimal execution, but since the view is not materialized by the database system, no view can even give a speedup in query execution.

5. Evaluation

To show that our implementation scales well with increasing query size, we designed several experiments. As already noted, umbra, the database system which we used for implementing our view matching algorithm, does not yet support materialized views, so we cannot offer an end-to-end benchmark that shows performance increase of detecting a matching materialized view. Instead, we show in several experiments that view matching is taking a fraction of the time required for semantic analysis, even for large queries.

We show that matching a single large view, matching many small views and detecting that a view does not match are all quick operations.

5.1. Large Views

To show that the view matching algorithm itself scales well with increasing query size, we designed the following experiment. We create a SQL string that corresponds to a random query tree, consisting of 2^k relations and forming a full balanced join tree with depth k . Each join is an equijoin between a random attribute from its left and right subtree, and each base table has between 1 and 40 attributes (equal distribution). An example query string for $k = 2$ is shown below.

```
1 SELECT *
2 FROM R1, R2, R3, R4
3 WHERE R1.a = R2.b AND R3.f = R4.d AND R1.b = R4.a
```

A corresponding query tree can be represented by the evaluation order $(R_1 \bowtie_{R_1.a=R_2.b} R_2) \bowtie_{R_1.b=R_4.a} (R_3 \bowtie_{R_3.f=R_4.d} R_4)$. Of course there could be other orders as well, but we guarantee that the balanced tree exists.

We create a view that is exactly equal to the query and then submit the query to the database system. In Figure 5.1, we show how much time our view matching implementation requires in comparison to the full semantic analysis. We can see that view matching is very fast, taking only a millisecond for $2^5 = 32$ relations, and then scales linearly with increasing query size. In comparison to the full semantic analysis it scales a bit better and for $2^{13} = 8192$ relations beats it by circa 2 orders of magnitude.

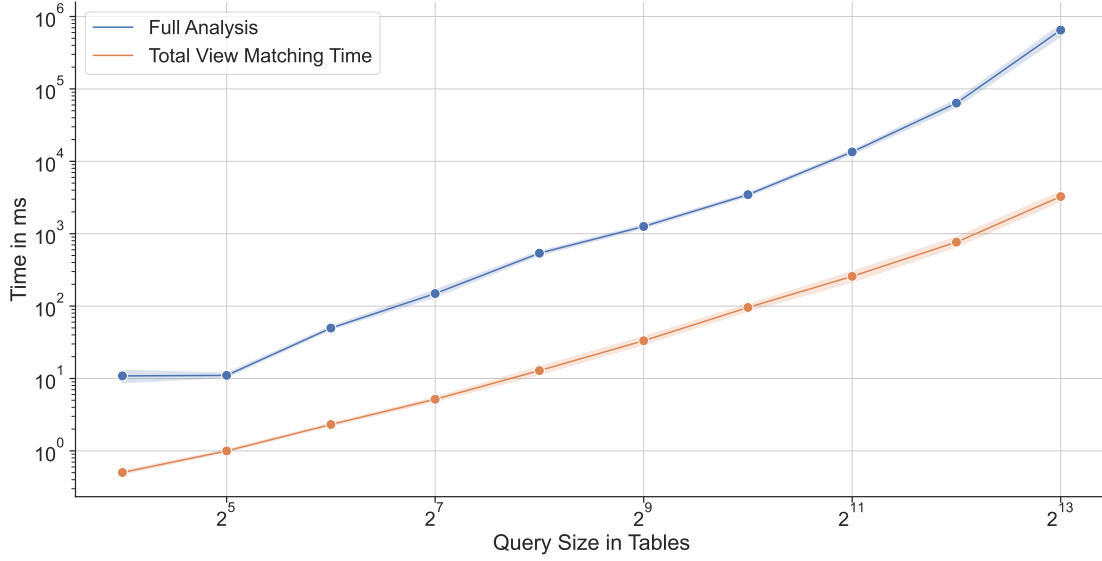


Figure 5.1.: Total view matching time for a random generated full balanced query tree with an equivalent view

The full query analysis takes around $800s = 13.3min$, while view matching only takes 5s. We performed the experiment 10 times for each size, generating a new random query in each iteration to minimize noise. The mean is plotted as a line and surrounded by the 95% confidence interval.

When we discussed the shortcomings of our implementation (cf. Section 4.4), we noted that we generate a separate query graph for view matching and also rebuild a query tree after view matching. The detailed breakdown of the full implementation is shown in Figure 5.2. In a perfect implementation where we would pass the query graph to join ordering immediately after performing view matching, the procedures for computing the query graph and view graph as well as rebuilding the query tree are obsolete. Thus, only the time for actual view matching is important, which is even three orders of magnitude lower than the time for full semantic analysis.

5.2. Multiple partially-matching Views

Since we especially tailored our view matching approach for partial view matching, we show that also matching multiple smaller views scales well. In the previous experiment, we proved that we have linear scaling for query size and one matching view. We did this by generating a random query tree and generated a view which is exactly equal.

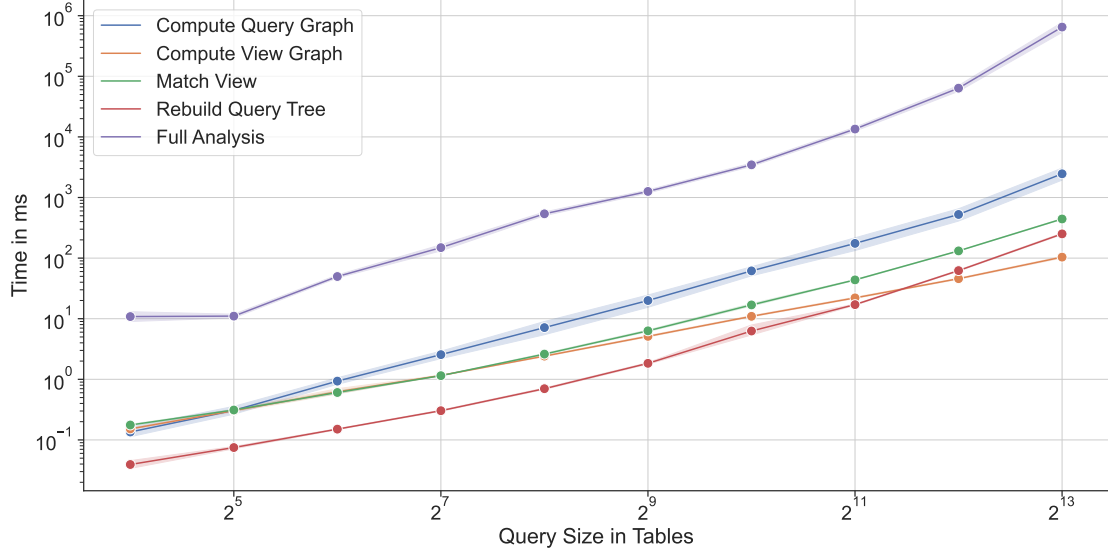


Figure 5.2.: Detailed View Matching time for a random generated fully balanced Query Tree with an equivalent View

In this experiment we fix the query size and show that matching multiple smaller views also scales well. We re-use the query tree from the previous experiment with, but instead of using an equivalent view, we select how many matching views we want to generate. The approach, depicted in Figure 5.3, is to split every view (starting with the full query) into two equally large subtrees and a join that connects both views. Since we constructed a full balanced tree, any tree with more than one join can be split.

We use 2^{10} relations, as a reasonably large query size and then try to match and insert 1 up to 128 views. The result is shown in Figure 5.4. We present the total view matching time, including building query graphs and rebuilding query trees and the time that the actual matching algorithm requires. We can see that the view matching part is increasing slowly to both sides with a sweet spot at 2^3 views. $2^3 = 8$ views formed by splitting a tree of 2^{10} relations consist of $2^7 = 128$ relations each. From this experiment it seems like matching multiple views is in quadratic complexity with a low slope, which is expected, since we have a linear view matching algorithm, applied N times for N views. Even for $2^7 = 128$ views, the view matching is 10 times faster than the overall semantic analysis.

What we can also see from the plot is, that the time for semantic analysis decreases with number of matching views. This is probably due to our simulation of materialized views with TempScan operators and query trees for the views below them. The join ordering algorithm will in our case also try to re-order joins inside each view, so for 1 view, on the far left of the graph, the time for join ordering in the view is exactly the

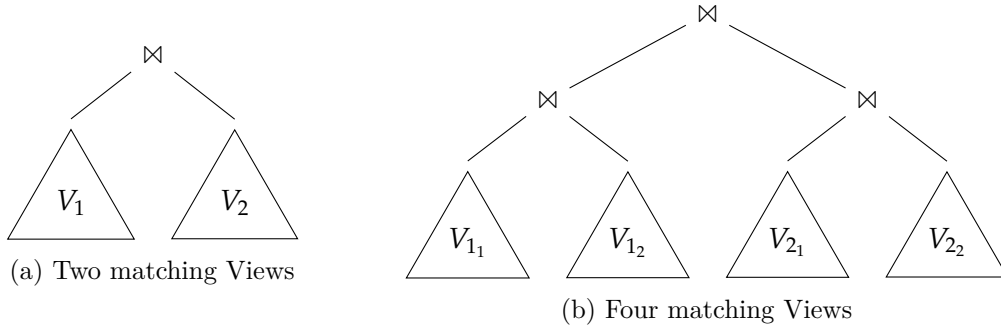


Figure 5.3.: Generating multiple non-overlapping Views from a Query Tree

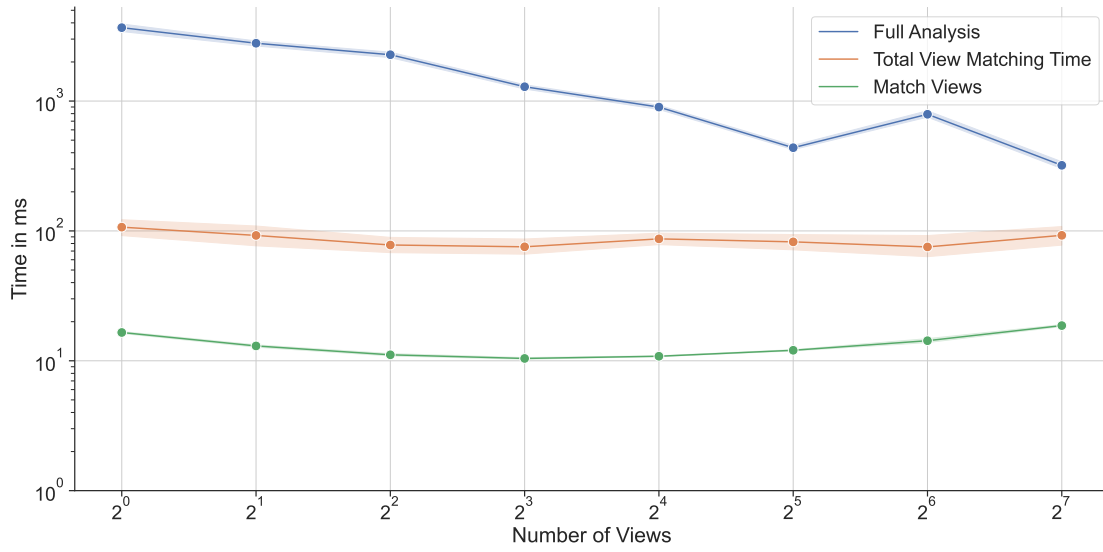


Figure 5.4.: View Matching time for a random generated fully balanced Query Tree with multiple matching Views

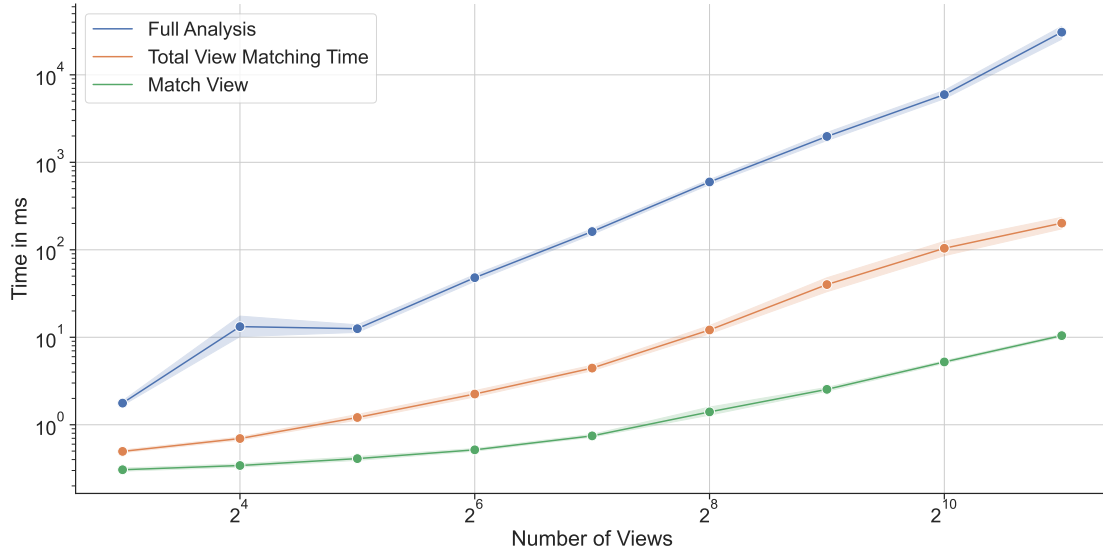


Figure 5.5.: View Matching time for non-matching Views

time join ordering takes in a query. The join-ordering problem is in NP, so solving less smaller problems is generally faster and reflected by the decreasing graph. For 2^5 views with 2^5 relations each, we seem to hit the sweet-spot for time to re-order joins in each view and amount of re-orderings performed.

cite

5.3. Non-matching Views

In the previous experiments, we showed that matching views is fast and scales well with increasing query sizes, as well as number of views. However, we specially tailored the views to match the query, so it remains to show that rejecting views is also fast and scales well.

In this experiment, we specifically tailor views to not match the query. There are multiple things we can do to make a view non-matching, such as introducing an additional relation or predicate, changing an existing predicate to be more restrictive or changing the type of a join from outer to inner or vice-versa.

We use the random generated query tree and a view that is equal to the tree, to then modify a random join predicate in the view from $a = b$ to $a = 2 \cdot b$. In Figure 5.5 we present the average and 95% confidence interval over 10 runs of view matching with a non matching view with respect to increasing query sizes. We can see that the time for checking that a view does not match is approximately the same as for a matching view.

This stems from the fact, the view matching will fail when it tries to check the

implication $a = 2 \cdot b \Rightarrow a = b$ which is false. To come to this check, the algorithm loops over all edges in a non-deterministic order and looks up the corresponding edge. There is a chance of $\frac{1}{|E_V|}$, that the non-matching edge will be checked first. The chance to pick it second is $(1 - \frac{1}{|E_V|}) \cdot \frac{1}{|E_V|-1}$. This experiment can be modeled as an urn problem without replacement. Edges will be drawn until the specific edge is picked and we are interested in the expected number of draws before the specific edge comes up. In general, if we have M edges which are more restrictive than their corresponding query edge and $N = |E_V|$ total edges, then the probability of checking X edges before finding a restrictive one is a random variable. X follows a negative hypergeometric distribution, and in our case since $M = 1$, the expected value for X is $\frac{|E_V|+1}{2}$. So, the algorithm will check about half of the edges until it stops. The green graph for actual view matching in Figure 5.5 is even lower than half of Figure 5.2, since the algorithm does not perform view insertion, which requires another loop over the query graph.

All in all, we can sum up that the algorithm scales well in any dimension for matching or rejecting multiple views on large queries.

6. Future Work

In this work, we presented a view matching algorithm for partial view matching on query graphs. We extended the approach to handle outer joins and non-SPOJ operators. In our implementation, we have shown that the algorithm scales very well for multiple views and large queries. Nevertheless, there are some topics that will require further research which we will discuss in the following.

First of all, we would like to extend the implementation with respect to nullification (Section 3.4.1) and attribute inclusion checks (Section 3.3.4). Also close to the implementation is the choice of predicate implication oracle. We discussed this in Section 4.2 and concluded that it is desirable to develop a well suited implication oracle that hits the sweet spot between complexity and completeness.

Considering more theoretical topics, we are interested in extending the handling of outer joins to join degradation (Section 3.4.2). Matching an inner join in a query with an outer join from a view would only require to match a hyperedge (R, S) in the view with an edge (r, s) in the query, where $r \subseteq R$ and $s \subseteq S$ and a less restrictive join predicate. It would not require nullification, because every tuple that is in the query was constructed from an inner join and all null tuples would not match the inner join and can be eliminated by the residual predicate. However, a proof is required to show that the approach works this straight forward.

Regarding non-SPOJ operators (Section 3.5), we suggest some more improvements. We introduced connecting operators into a graph of SPOJ-components to handle all non-SPOJ operators and match them if they have the same input subtrees and expressions where the input subtrees can differ in join ordering in query and view. This restriction was introduced, because in general feeding a different input set to an arbitrary operator could lead to an output that is not easily convertible to the original output. So the approach can handle any operator that can be implemented in a database system. However, its' accuracy could be greatly improved by introducing special handling for each operator, or at least some of the operators. We present several examples:

Anti-joins are similar to outer joins, because they can be represented in the query graph with their re-ordering constraints. The main difference for view matching is that the result of $R \bowtie S$ grows bigger, the smaller S is. So to ensure tuple inclusion, we would need to check that a view has less tuples in S than the query. This could be

enforced if we reverse the predicate implication to $p(e_v) \Rightarrow p(e_q)$ for all predicates in S .

Union operations $R \cup S$ could be handled better if the schema of their inputs is disjunct $\mathcal{S}(R) \cap \mathcal{S}(S) = \emptyset$. Let the view be $R' \cup S$, where $R \subseteq R'$, that is R contains less tuples than R' . The additional tuples of R' can be filtered out with a residual predicate $p_{\bar{r}}$. However, if the residual predicate is applied after the union, care must be taken to not accidentally filter out a tuple that is a null-extended tuple of S : $(null_{\mathcal{S}(R)}) \cup s$ where $s \in S$. So the residual predicate must be extended to $p_{\bar{r}} \vee (\forall a \in \mathcal{S}(R) : a = null)$ to keep all tuples that come from S . Therefore, R must not contain the null tuple.

Aggregations can be improved in several ways. For example a grouping by a set of attributes A , that aggregates attributes A' does not modify any attribute in A . So if a query does not contain an aggregation and refers only to attributes in A , a view with an aggregation that groups by A can be inserted, by just ignoring the aggregation.

Another interesting improvement is combining aggregation expressions. If a view contains an aggregation that contains $s = sum(x)$ and $c = count(x)$, then a query with an aggregation with the same inputs but expression $a = avg(x)$ could be replaced by the view with an inserted expression of $a = s/c$. This can be generalized to other expressions.

Another improvement towards better view matching in general would be a standardized benchmark, that contains views and queries and measures how many matching views could be detected. This way, the completeness and false rejection rates of view matching algorithms can be assessed. Together with their runtime- and implementation complexity, different approaches could then be compared quantitatively, leading to the development of better view matching algorithms.

7. Related Work

While our approach to view matching is novel in terms of using the query graph with outer joins, other work with regard to view matching has been conducted in the past. We will highlight some of the relevant results and set them in context to our work.

A good overview regarding materialized views in general is given in [21]. They discuss the various problems of view selection, view maintenance and view matching and present several existing solutions.

For a more detailed view on view selection refer to [13]. If the view selection algorithm chooses a view to be materialized for a given query in a workload, then the view must match the query. In these cases, we could pre-compute the residual predicate and reduce view matching to the insertion step. However, view selection in general is NP-hard. In fact, a greedy heuristic to minimize average query execution time of a given workload gives no performance guarantees at all [12]. This implies, that probably also sub-optimal views will be materialized. The optimizer will have to decide which of the available views to apply, so it is better to consider all views as we did in our DP-based plan generator, than rely on another heuristic that operates on already sub-optimal views.

View maintenance is relevant to the view matching problem, because the views have to be held up to date, for example by incremental maintenance [14, 15]. Techniques to extend this approach for efficient updating of materialized views that contain outer joins were developed [22], so it makes sense to discuss outer-join view matching.

In [17] an overview over different approaches to view matching is given. The most notable distinction is between view matching for query optimization and for data integration. Data integration provides a uniform query interface to heterogenous data sources represented as views, while query optimization tries to find a cost-based optimal rewrite of a query with views to optimize execution times. Our approach falls in the latter category and we also show how to integrate it into a dynamic programming based plan generator. Finding all possible or optimal rewritings of queries using views is NP-complete [2]. We hide this complexity in the DP-based plan generator, since plan generation itself is also NP complete. Another benefit of including view matching there is that as Agrawal et al. showed, view matching is closely connected to index selection [10]. When a view lacks an index of a base-table that would be highly beneficial to query evaluation, it might be better to not introduce the view and evaluate the original query. This can all be handled by a well defined cost function for a view aware plan generator.

Goldstein et al. present a scalable algorithm for SPJG views, that is extended by Larson et al. to outer-join matching [4, 16]. They only handle views with a single group-by on top and their SPOJG view matching requires the minimum union operation. In comparison, our approach uses the query graph to encode join re-ordering constraints of outer join queries instead of relying on minimum unions. Also, we can handle multiple group-by statements, however without tuple reconstruction.

More extensive techniques for tuple recovery from aggregations is given in [23]. Thus, they improve matching for views and queries that contain aggregations from syntactic matching towards semantic. Cohen et al. formalized the rewrites for aggregation functions in general [24]. We hope to merge these techniques into our connecting operators for aggregations as discussed in Chapter 6.

Graph based view matching already exists, for example bottom-up on ASTs [25]. While Zaharioudakis et al. also use a “compensation” process like we did with nullification and residual predicate, they ignore different join orderings. Bello et al. also perform view matching on the query graph (or as they call it join graph) with handling of aggregations [26]. It is unclear how they handle outer-, anti- and semi-joins since they do not seem to use hyperedges to represent them in the query graph, and also do not provide a procedure for nullification. Moreover, they only heuristically apply views and then compare the optimized execution plan with views to the original optimized execution plan and choose the better option. In contrast, our integration into the DP-based plan generator selects the views to apply dynamically and can guarantee optimal plans.

Several view matching approaches also exist for non-SQL based systems, such as Datalog [27]. The approach of Afrati et al. searches for execution plans that only use materialized views. They develop different cost models and evaluate corresponding heuristics to keep the search space small. Chaudhuri et al. [28] generate a MapTable, that maps Datalog expressions to equivalent views. By extending the DP-table, we build a similar MapTable, but while Chaudhuri et al. can generate this statically from a view, we need to generate the DP-table for any query-view combination.

A way for partial view matching of queries in object oriented query language OQL is presented in [29]. They assert a normal form that includes having all predicates in CNF and no subqueries (thus outer joins are impossible to represent) and match predicates with implication similar to our approach.

8. Conclusion

In this work, we presented an algorithm to solve the partial view matching problem. The problem is to find an optimal execution plan of a query, that can potentially use any materialized view in the system. A speedup is for example achieved, when costly computations can be avoided by substituting them with materialized views instead. We split that problem up into two parts, detecting whether a view matches and making the plan generator view aware. Our algorithm for checking the match is based on query graphs. They are especially well suited to this problem, since query graphs are independent of join-ordering and will be computed by the optimizer regardless of view matching.

The algorithm first checks the inclusion of the view graphs vertices in the query graph, to find a subgraph that the view could replace. It then checks whether all required tuples are available in the view by predicate matching. The predicate matching is extendable, such that users can tweak the accuracy of view matching to their needs. Higher accuracy in this case means more complexity and less speed. Lastly, the algorithm checks attribute inclusion, with respect to equivalence classes. Overall, our approach to view matching is in linear time with respect to query size

We have also shown how to adapt the algorithm to handle outer joins and even arbitrary non-SPOJ operators. While non-SPOJ operators are currently matched syntactically but oblivious of join ordering, our approach is extendable in a way, that each operator could have its own matching rule, with a fallback to syntactic matching. For example, we discussed potential extensions for anti-joins, unions and aggregations. Again such extensions would potentially increase the applicability of a view, but increase time complexity, as matching becomes more sophisticated.

We implemented our algorithm and then showed in various experiments that it indeed scales as well as discussed. Furthermore, we explained how to integrate the approach in a DP-based plan generator.

All in all, we presented a full front-to-back solution for partial view matching, that is well suited for integration in an existing database system by re-using existing data structures, offering tweakable

A. Proofs

In the following we give the proofs for the equivalences we stated in Section 2.1.

Lemma 1. *Cross products are commutative, $R \times S = S \times R$.*

Proof.

$$\begin{aligned}
 R \times S &= \{r \cup s \mid r \in R \wedge s \in S\} \\
 &= \{(r_1, \dots, r_n, s_1, \dots, s_m) \mid (r_1, \dots, r_n) \in R \wedge (s_1, \dots, s_m) \in S\} \\
 &= \{(s_1, \dots, s_m, r_1, \dots, r_n) \mid (r_1, \dots, r_n) \in R \wedge (s_1, \dots, s_m) \in S\} \\
 &= \{s \cup r \mid r \in R \wedge s \in S\} \\
 &= S \times R
 \end{aligned}$$

The equivalence in line 3 holds, since attributes are named and we work with unordered tuples. In fact we could directly use that $s \cup r = r \cup s$. $\square$

Lemma 2. *Joins are commutative, $R \bowtie_p S = S \bowtie_p R$.*

Proof.

$$\begin{aligned}
 R \bowtie_p S &= \sigma_p (R \times S) \\
 &= \sigma_p (S \times R) \\
 &= S \bowtie_p R
 \end{aligned}$$

$\square$

Lemma 3. *Projections can be unified $\Pi_\beta \Pi_\gamma R = \Pi_{\beta \cap \gamma} R$.*

Proof.

$$\begin{aligned}
 \Pi_\beta \Pi_\gamma R &= \{t_\beta \mid t \in \Pi_\gamma R\} \\
 &= \{t_\beta \mid t \in \{t'_\gamma \mid t' \in R\}\} \\
 &= \{t_{\beta \cap \gamma} \mid t \in R\} \\
 &= \Pi_{\beta \cap \gamma} R
 \end{aligned}$$

$\square$

Corollary 4. *Projections are commutative, $\Pi_\beta \Pi_\gamma R = \Pi_{\beta \cap \gamma} R = \Pi_\gamma \Pi_\beta R$.*

Lemma 5. *Selections can be unified, $\sigma_p \sigma_q R = \sigma_{p \wedge q} R$.*

Proof.

$$\begin{aligned} \sigma_p \sigma_q R &= \{r \mid r \in \sigma_q R \wedge p(r)\} \\ &= \{r \mid r \in \{r' \mid r' \in R \wedge q(r')\} \wedge p(r)\} \\ &= \{r \mid r \in R \wedge q(r) \wedge p(r)\} \\ &= \sigma_{p \wedge q} R \end{aligned}$$

□

Corollary 6. *Selections are commutative, $\sigma_p \sigma_q R = \sigma_{q \wedge p} R = \sigma_q \sigma_p R$.*

Lemma 7. *Projections can be moved in front of Selections, $\sigma_p (\Pi_\beta R) = \Pi_\beta (\sigma_p R)$. Note that the opposite does not hold, since p might reference an attribute $t \notin \beta$, so the Selection can not be applied after the Projection.*

Proof.

$$\begin{aligned} \sigma_p (\Pi_\beta R) &= \{r \mid r \in \Pi_\beta R \wedge p(r)\} \\ &= \{r \mid r \in \{r'_\beta \mid r' \in R\} \wedge p(r)\} \\ &= \{r_\beta \mid r \in R \wedge p(r)\} \\ &= \Pi_\beta (\sigma_p R) \end{aligned}$$

□

Lemma 8. *Projections and Selections can be moved out of cross products, $(\Pi_\beta R) \times (\sigma_q S) = \Pi_{\beta \cup S(S)} \sigma_q (R \times S)$*

Proof.

$$\begin{aligned} (\Pi_\beta R) \times (\sigma_q S) &= \{r \cup s \mid r \in (\Pi_\beta R) \wedge s \in (\sigma_q S)\} \\ &= \{r \cup s \mid r \in \{t_\beta \mid t \in R\} \wedge s \in \{s' \mid s' \in S \wedge q(s')\}\} \\ &= \{r \cup s \mid r \in \{t_\beta \mid t \in R\} \wedge s \in S \wedge q(s)\} \\ &= \{r_\beta \cup s \mid r \in R \wedge s \in S \wedge q(s)\} \\ &= \Pi_{\beta \cup S(S)} \sigma_q (R \times S) \end{aligned}$$

□

Lemma 9. *Projections and Selections can be moved out of joins, $(\Pi_\beta R) \bowtie_p (\sigma_q S) = \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q (R \bowtie_p S)$*

Proof.

$$\begin{aligned}
 (\Pi_\beta R) \bowtie_p (\sigma_q S) &= \sigma_p ((\Pi_\beta R) \times (\sigma_q S)) \\
 &= \sigma_p \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q (R \times S) \\
 &= \Pi_{\beta \cup \mathcal{S}(S)} \sigma_p \sigma_q (R \times S) \\
 &= \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q \sigma_p (R \times S) \\
 &= \Pi_{\beta \cup \mathcal{S}(S)} \sigma_q (R \bowtie_p S)
 \end{aligned}$$

□

In Section 3.3.3, we gave the intuition that the output of a query graph with the same vertices only depends on the included predicates. We will now prove this formally.

Lemma 10. *Let $G = (V, E) = (\{V_1, \dots, V_n\}, \{e_1, \dots, e_m\})$ be a query graph over the relations $V_1, \dots, V_n$, then $\text{output}(G) = \sigma_{\bigwedge_{i=1}^m p(e_i)} (V_1 \times \dots \times V_n)$*

Proof. We will prove the claim by induction. For $G = (\{V_1\}, \emptyset)$, the claim trivially holds, since the output of the graph is V_1 . Assume that the hypothesis holds for any graph with $|V| \leq n$.

Then for $|V| = n + 1$, we have a graph $G = (\{V_1, \dots, V_n, V_{n+1}\}, E)$. If $E = \emptyset$, then the claim already holds, since the output of the graph will be $V_1 \times \dots \times V_{n+1}$. If E only contains self-loops e_i and $p(e_i) = \text{true}$ if $e_i \notin E$, then the output of the graph will be $\sigma_{p(e_1)} V_1 \times \dots \times \sigma_{p(e_{n+1})} V_{n+1} = \sigma_{p(e_1)} \dots \sigma_{p(e_{n+1})} (V_1 \times \dots \times V_{n+1}) = \sigma_{\bigwedge_{i=1}^m p(e_i)} (V_1 \times \dots \times V_{n+1})$ so the claim holds.

So there must be an edge $e \in E$ that connects at least two vertices. Let this edge connect the vertices $V_{n+1-k}, \dots, V_{n+1}$. According to the rebuild procedure, we can pick this edge and unify the respective vertices into V_x with an associated query tree $\sigma_{p(e)} (V_{n+1-k} \times \dots \times V_{n+1})$. Let the rest of the graph be $G_y = (\{V_1, \dots, V_{n-k}\}, E')$ with $E' = \{e_1, \dots, e_{m'}\}$. Per induction hypothesis, we know that $\text{output}(G_y) = \sigma_{\bigwedge_{i=1}^{m'} p(e_i)} (V_1 \times \dots \times V_{n-k})$. Thus, we can also apply the rebuild algorithm on G_y to obtain a node V_y with an associated query tree that is equivalent to the expression.

Our remaining graph has nodes $(\{V_x, V_y\})$ and either a single edge that connects both vertices, or no edge. Assume there is an edge e_r with associated predicate p_r . As discussed in the rebuild procedure e_r must have been merged from all edges that are between V_x and G_y . These edges are $E \setminus (E' \cup e)$ and p_r must be the conjunction of all predicates that were associated with them, so $p_r = \bigwedge_{e_i \notin (E' \cup e)} p(e_i)$.

We can then pick the edge e_r and rebuild the following query tree:

$$\begin{aligned}
& \sigma_{p_r} ((\sigma_{p(e)}(V_{n+1-k} \times \dots \times V_{n+1})) \times (\sigma_{\bigwedge_{i=1}^{m'} p(e_i)}(V_1 \times \dots \times V_{n-k}))) \\
&= \sigma_{p_r} \sigma_{p(e)} \sigma_{\bigwedge_{i=1}^{m'} p(e_i)} ((V_{n+1-k} \times \dots \times V_{n+1}) \times (V_1 \times \dots \times V_{n-k})) \\
&= \sigma_{\bigwedge_{i=1}^{m'} p(e_i) \wedge p(e) \wedge p_r} (V_1 \times \dots \times V_{n+1}) \\
&= \sigma_{\bigwedge_{i=1}^{m'} p(e_i) \wedge_{e_i \notin (E' \cup e)} p(e_i) \wedge p(e)} (V_1 \times \dots \times V_{n+1}) \\
&= \sigma_{\bigwedge_{i=1}^m p(e_i)} (V_1 \times \dots \times V_{n+1})
\end{aligned}$$

If there is no edge e_r , then choose $p_r = \text{true}$, and the observation will hold, since in this case $m' + 1 = m$. So $\text{output}(G) = \sigma_{\bigwedge_{i=1}^m p(e_i)} (V_1 \times \dots \times V_n)$. $\square$

List of Figures

2.1. Translating a Query to a Query Tree	8
2.2. Converting a Query Graph into a Join Tree	10
2.3. Ambiguity of Natural Extension of a Query Graph with Outer Joins . . .	11
2.4. Extension of a Query Graph by Outerjoin re-ordering Constraints as Hyperedges	11
3.1. Example of view matching on Query Trees	18
3.2. Example of failing view matching on Query Trees	18
3.3. A graph representation of the view matching that failed to match in the trivial query tree algorithm	20
3.4. Inserting a View Graph into a Query Graph	25
3.5. Query Graph that contains more Edges and other predicates wrt. the View Graph	25
3.6. Outer Joins with Predicate Implication	33
3.7. Example of matching view with aggregation	39
3.8. Data Structures for non-SPOJ Query	40
3.9. View matching with Connecting Operators and Subgraphs	41
5.1. Total view matching time for a random generated full balanced query tree with an equivalent view	50
5.2. Detailed View Matching time for a random generated fully balanced Query Tree with an equivalent View	51
5.3. Generating multiple non-overlapping Views from a Query Tree	52
5.4. View Matching time for a random generated fully balanced Query Tree with multiple matching Views	52
5.5. View Matching time for non-matching Views	53

Bibliography

- [1] T. Neumann and B. Radke, “Adaptive optimization of very large join queries,” in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 677–692.
- [2] A. Y. Levy, A. O. Mendelzon, and Y. Sagiv, “Answering queries using views,” in *Proceedings of the fourteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, 1995, pp. 95–104.
- [3] C. A. Galindo-Legaria, “Outerjoins as disjunctions,” in *Proceedings of the 1994 ACM SIGMOD international conference on Management of data*, 1994, pp. 348–358.
- [4] P.-Å. Larson and J. Zhou, “View matching for outer-join views,” *The VLDB Journal*, vol. 16, no. 1, pp. 29–53, Jan. 2007, Place: Berlin, Heidelberg Publisher: Springer-Verlag, ISSN: 1066-8888. DOI: 10.1007/s00778-006-0027-9.
- [5] C. Galindo-Legaria and A. Rosenthal, “Outerjoin simplification and reordering for query optimization,” *ACM Transactions on Database Systems*, vol. 22, no. 1, pp. 43–74, Mar. 1997, ISSN: 0362-5915, 1557-4644. DOI: 10.1145/244810.244812.
- [6] G. Moerkotte. (Feb. 12, 2020). Building query compilers (draft), [Online]. Available: <https://pi3.informatik.uni-mannheim.de/~moer/querycompiler.pdf>.
- [7] A. Rosenthal and C. Galindo-Legaria, “Query graphs, implementing trees, and freely-reorderable outerjoins,” in *Proceedings of the 1990 ACM SIGMOD international conference on Management of data*, 1990, pp. 291–299.
- [8] G. Moerkotte and T. Neumann, “Dynamic programming strikes back,” in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, 2008, pp. 539–552.
- [9] J. Rao, B. Lindsay, G. Lohman, H. Pirahesh, and D. Simmen, “Using EELs, a practical approach to outerjoin and antijoin reordering,” in *Proceedings 17th International Conference on Data Engineering*, 2001, pp. 585–594. DOI: 10.1109/ICDE.2001.914873.
- [10] S. Agrawal, S. Chaudhuri, and V. R. Narasayya, “Automated selection of materialized views and indexes in SQL databases,” in *Proceedings of the 26th International Conference on Very Large Data Bases*, ser. VLDB ’00, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, pp. 496–505, ISBN: 1-55860-715-3.

- [11] A. Kemper and A. Eickler, *Datenbanksysteme. Eine Einführung. 10., aktualisierte und erweiterte Auflage*, 10th ed. Oldenbourg Wissenschaftsverlag GmbH, München, 2015, ISBN: 978-3-11-044375-2.
- [12] H. Karloff and M. Mihail, “On the complexity of the view-selection problem,” in *Proceedings of the eighteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, 1999, pp. 167–173.
- [13] I. Mami and Z. Bellahsene, “A survey of view selection methods,” *Sigmod Record*, vol. 41, pp. 20–29, 2012. DOI: 10.1145/2206869.2206874.
- [14] A. Gupta, I. S. Mumick, *et al.*, “Maintenance of materialized views: Problems, techniques, and applications,” *IEEE Data Eng. Bull.*, vol. 18, no. 2, pp. 3–18, 1995, Publisher: Citeseer.
- [15] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, “Maintaining views incrementally,” *ACM SIGMOD Record*, vol. 22, no. 2, pp. 157–166, 1993, Publisher: ACM New York, NY, USA.
- [16] J. Goldstein and P.-Å. Larson, “Optimizing queries using materialized views: A practical, scalable solution,” *SIGMOD Rec.*, vol. 30, no. 2, pp. 331–342, May 2001, Place: New York, NY, USA Publisher: Association for Computing Machinery, ISSN: 0163-5808. DOI: 10.1145/376284.375706.
- [17] A. Y. Halevy, “Answering queries using views: A survey,” *The VLDB Journal*, vol. 10, no. 4, pp. 270–294, 2001, Publisher: Springer.
- [18] T. Neumann and A. Kemper, “Unnesting arbitrary queries,” *Datenbanksysteme für Business, Technologie und Web (BTW 2015)*, 2015, Publisher: Gesellschaft für Informatik eV.
- [19] D. C. Kozen, “Union-find,” in *The Design and Analysis of Algorithms*, New York, NY: Springer New York, 1992, pp. 48–51, ISBN: 978-1-4612-4400-4. DOI: 10.1007/978-1-4612-4400-4_10.
- [20] C. Galindo-Legaria and M. Joshi, “Orthogonal optimization of subqueries and aggregation,” *ACM SIGMOD Record*, vol. 30, no. 2, pp. 571–581, 2001, Publisher: ACM New York, NY, USA.
- [21] R. Chirkova and J. Yang, “Materialized views,” *Foundations and Trends in Databases*, vol. 4, no. 4, pp. 295–405, 2011, Publisher: Citeseer.
- [22] A. Nica, “Immediate materialized views with outerjoins,” in *Proceedings of the ACM 13th International Workshop on Data Warehousing and OLAP*, ser. DOLAP ’10, event-place: Toronto, ON, Canada, New York, NY, USA: Association for Computing Machinery, 2010, pp. 45–52, ISBN: 978-1-4503-0383-5. DOI: 10.1145/1871940.1871951.

- [23] D. Srivastava, S. Dar, H. Jagadish, and A. Levy, “Answering queries with aggregation using views,” Jan. 1997.
- [24] S. Cohen, W. Nutt, and Y. Sagiv, “Rewriting queries with arbitrary aggregation functions using views,” *ACM Trans. Database Syst.*, vol. 31, no. 2, pp. 672–715, Jun. 2006, Place: New York, NY, USA Publisher: Association for Computing Machinery, ISSN: 0362-5915. DOI: 10.1145/1138394.1138400.
- [25] M. Zaharioudakis, R. Cochrane, G. Lapis, H. Pirahesh, and M. Urata, “Answering complex SQL queries using automatic summary tables,” in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’00, event-place: Dallas, Texas, USA, New York, NY, USA: Association for Computing Machinery, 2000, pp. 105–116, ISBN: 1-58113-217-4. DOI: 10.1145/342009.335390.
- [26] R. G. Bello, K. Dias, A. Downing, J. Feenan, J. Finnerty, W. D. Norcott, H. Sun, A. Witkowski, and M. Ziauddin, “Materialized views in oracle,” in *VLDB*, vol. 98, 1998, pp. 24–27.
- [27] F. N. Afrati, C. Li, and J. D. Ullman, “Generating efficient plans for queries using views,” *SIGMOD Rec.*, vol. 30, no. 2, pp. 319–330, May 2001, Place: New York, NY, USA Publisher: Association for Computing Machinery, ISSN: 0163-5808. DOI: 10.1145/376284.375705.
- [28] S. Chaudhuri, R. Krishnamurthy, S. Potamianos, and K. Shim, “Optimizing queries with materialized views,” in *Proceedings of the Eleventh International Conference on Data Engineering*, Mar. 1995, pp. 190–200. DOI: 10.1109/ICDE.1995.380392.
- [29] D. Florescu, L. Raschid, and P. Valduriez, “Answering queries using OQL view expressions,” *Workshop on Materialized Views, in cooperation with ACM SIGMOD*, Jul. 1996.